Zarządzanie w systemach i sieciach komputerowych

Dr inż. Robert Wójcik

Wykład 2. Zasoby i procesy w systemach i sieciach komputerowych

2.1. Systemy i sieci komputerowe – zasoby i procesy

2.2. Zarządzanie zasobami i procesami

2.3. Zarządzanie wątkami

2.4. Modele obliczeń

2.1. Systemy i sieci komputerowe – zasoby i procesy

http://wazniak.mimuw.edu.pl/index.php - systemy operacyjne, sieci komputerowe.

Zasoby – składowe systemu komputerowego, sieci komputerowej.

Składowe sprzętowe: pamięć, procesory, magistrale komunikacyjne, dyski, drukarki i inne urządzenia peryferyjne, media transmisji danych (np. kable sieciowe), routery, przełączniki, punkty dostępu bezprzewodowego, itp.

Składowe programowe: system operacyjny, oprogramowanie użytkowe i systemowe, usługi sieciowe.

Zasoby informacyjne: pliki, bazy danych, zasoby internetowe.

Zasobem jest każdy element systemu, który może okazać się niezbędny dla realizacji przetwarzania. Typowe zasoby kojarzone są z elementami sprzętowymi systemu komputerowego. Jednak to dopiero system operacyjny definiuje konkretny element jako zasób, gdyż w jądrze systemu istnieją struktury danych wykorzystywane do zarządzania zasobami, a także procedury realizacji przydziałów oraz odzyskiwania zasobów.

Niektóre zasoby są tworzone przez jądro systemu operacyjnego na poziomie logicznym. Zasoby takie często określa się jako *wirtualne*. Przykładem wirtualnego urządzenia wejścia-wyjścia jest *plik*. Pliki udostępnia system operacyjny. Na poziomie sprzętowym można co najwyżej mówić o sektorach dysku, w których składowana są dane zawarte w pliku.

System operacyjny

System operacyjny jest warstwą oprogramowania operującą bezpośrednio na sprzęcie, której celem jest zarządzanie zasobami systemu komputerowego.

System operacyjny stwarza użytkownikowi środowisko ułatwiające wykorzystanie zasobów sprzętowych, programowych i informacyjnych.

Definicja procesu

Proces (ogólnie) – sekwencja zmian dokonujących się według określonego algorytmu.

Proces (systemy operacyjne) - elementarna jednostka pracy (aktywności) zarządzana przez system operacyjny, która ubiega się o zasoby systemu komputerowego w celu wykonania programu w sposób sekwencyjny, tj. w dowolnym momencie na zamówienie danego procesu może być wykonywany co najwyżej jeden rozkaz kodu programu.

• Proces sekwencyjny = wykonujący się program.

• Elementy składowe procesu:

– program — definiuje zachowanie procesu,
– dane — zbiór wartości przetwarzanych oraz wyniki,
– zbiór zasobów tworzących środowisko wykonawcze,
– blok kontrolny procesu (PCB, deskryptor) — opis bieżącego stanu procesu (stos procesu, dane tymczasowe - parametry procedur, adresy powrotu i zmienne tymczasowe).

Proces jest obiektem dynamicznym, który służy do organizowania wykonywania pasywnego programu w ten sposób, że definiuje on powiązanie niezbędnych zasobów systemu komputerowego w celu realizacji określonych operacji, a także umożliwia kontrolę stanu tych zasobów w ramach wykonywanego programu.

Różnica pomiędzy procesem a programem

Program jest zbiorem instrukcji, czyli tylko pasywnym elementem procesu, znajdującym się w jego segmencie kodu (zwanym też segmentem tekstu).

Do wykonania programu potrzebne są dodatkowe zasoby (procesor, pamięć itp.). Program najczęściej nie zmienia się w czasie wykonywania (nie ulega modyfikacji; z wyjątkiem dynamicznych metaprogramów pisanych w metajęzykach programowania, które umożliwiają tzw. metaprogramowanie, tj. umożliwiają programom tworzenie lub modyfikację kodu innych programów lub ich samych, np. takie możliwości dają języki Javascript, Lisp, Perl, PHP, Prolog, Python, Ruby, Groovy, Smalltalk), podczas gdy stan procesu ulega zmianie: zmienia się stan wykonywania programu podobnie jak stan większości zasobów wymaganych do jego realizacji.

W wyniku wykonywania procesu zmianie ulega np. segment danych, segment stosu, stan rejestrów procesora itp. Elementy te stanowią kontekst definiujący proces i są niezbędne do wykonania programu.

Współbieżność przetwarzania i współdzielenie zasobów

Wyodrębnienie procesu wiąże się ze współbieżnością przetwarzania oraz współdzieleniem zasobów. Działanie współbieżne oznacza, że zdolność obliczeniowa procesora (procesorów) ulega podziałowi między poszczególne procesory i jest realizowana poprzez przełączanie procesów (wzrost wydajności komputera, jeśli czas przełączania nie jest zbyt kosztowny).

W systemie może istnieć wiele procesów (wiele niezależnych przetwarzań), które korzystają z tych samych zasobów. Stąd ważne jest utrzymanie informacji o tym, które zasoby są przydzielone na potrzeby każdego przetwarzania, a także odpowiednie zarządzanie dostępem do zasobów.

Tworzenie i usuwanie procesów - elementarna komunikacja międzyprocesowa

Operacje tworzenia i usuwania procesów bezpośrednio dotyczą procesów, ale pośrednio również często zasobów, gdyż utworzenie procesu wymaga przydziału pewnych zasobów. Proces tworzony jest przez inny proces i początkowo może współdzielić z nim większość zasobów.

Przydział i zwalnianie jednostek zasobów

Operacje przydziału i zwalniania jednostek zasobów dotyczą tworzenia powiązań między procesami i zasobami.

Elementarne operacje wejścia-wyjścia

Elementarne operacje wejścia-wyjścia dotyczą również zasobów. Nie są to operacje związane z ich przydziałem, czy zwalnianiem, ale operacje dostępu do przydzielonych zasobów.

Nie wszystkie operacje dostępu do przydzielonych zasobów wymagają wsparcia ze strony jądra. Na przykład operacje dostępu do pamięci są realizowane na poziomie maszynowym (sprzętowym, np. DMA), jądro natomiast angażowane jest dopiero w przypadku wykrycia jakichś nieprawidłowości.

Procedury obsługi przerwań

Procedury obsługi przerwań z kolei są reakcją na zdarzenia zewnętrzne lub pewne szczególne stany wewnętrzne, które mogą być skutkiem ubocznym realizacji dostępu do zasobów. Reakcja na przerwanie może prowadzić do zmiany stanu procesu lub zasobu, nie zawsze jednak taka zmiana jest bezpośrednio spowodowana wykonaniem procedury obsługi. Procedury obsługi przerwań wykonywane muszą być szybko, dlatego ich bezpośrednim skutkiem jest czasami tylko odnotowanie faktu zajścia zdarzenia, natomiast właściwa reakcja systemu, w konsekwencji, której nastąpi zmiana stanu procesu lub zasobu, wykonywana jest później, np. przez inny proces, który monitoruje cyklicznie licznik wystąpienia zdarzenia.

W niektórych systemach komputerowych do obsługi zdarzeń, zamiast mechanizmu przerwań, wykorzystywane jest podejście oparte o cykliczne monitorowanie stanu i odpytywanie urządzeń, realizowane w oparciu o z góry zadany cykliczny harmonogram.

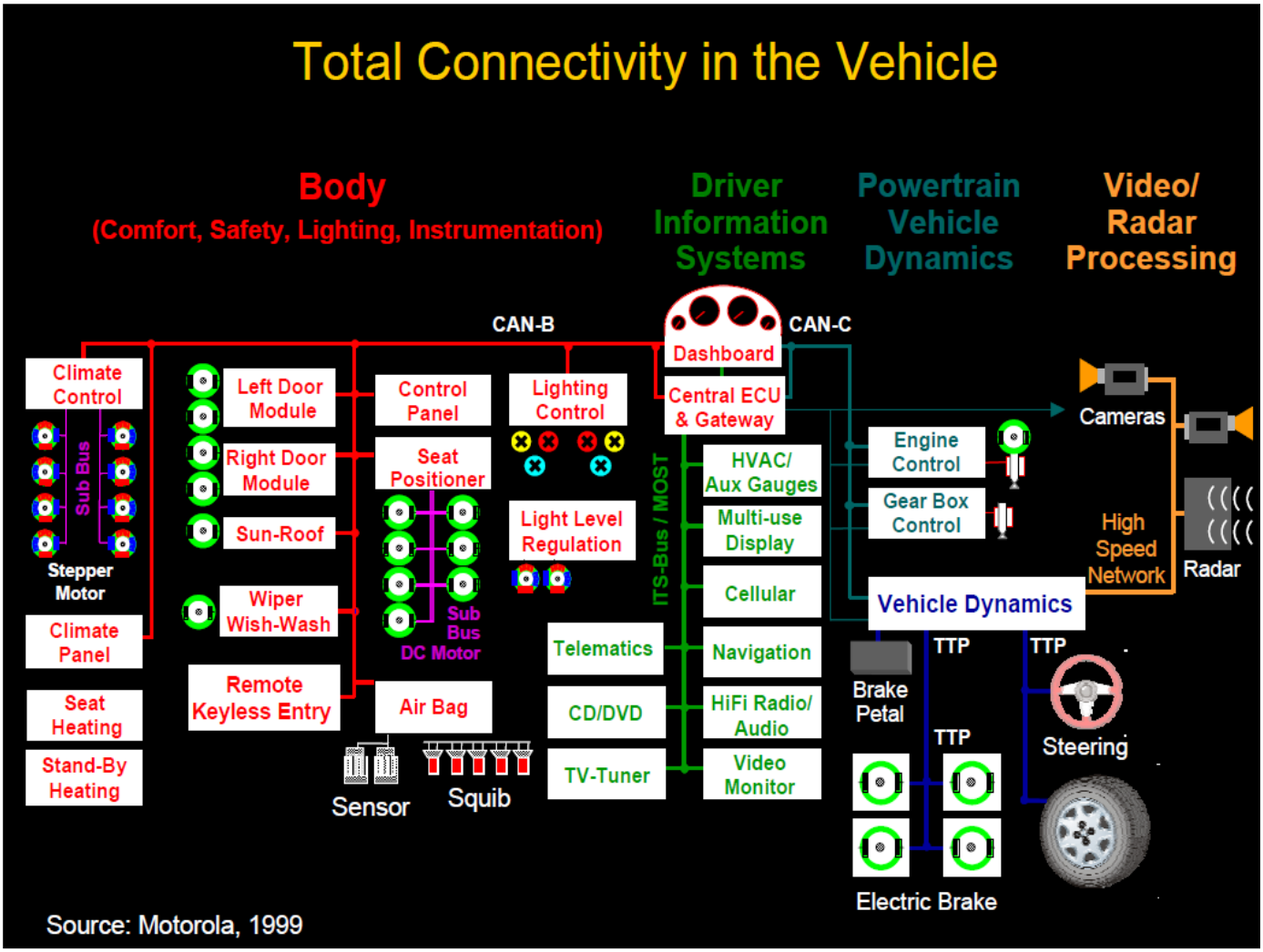
Architektury wyzwalane czasem TTA (Time-Triggered Architectures)

W systemach komputerowych wyzwalanych czasem TTA, operacje każdego procesu są wyzwalane przez globalny, cykliczny zegar (zamiast wykorzystywania przerwań), a poszczególne procesy cykliczne (np. związane z monitorowaniem stanu czujników, sterowaniem) komunikują się poprzez wspólną szynę danych, która w danej chwili może przesyłać tylko jedną wiadomość (np. systemy sterowania w przemyśle samochodowym, kolejnictwie, lotnictwie).

Sterowniki klasy TT (Time-Triggered)

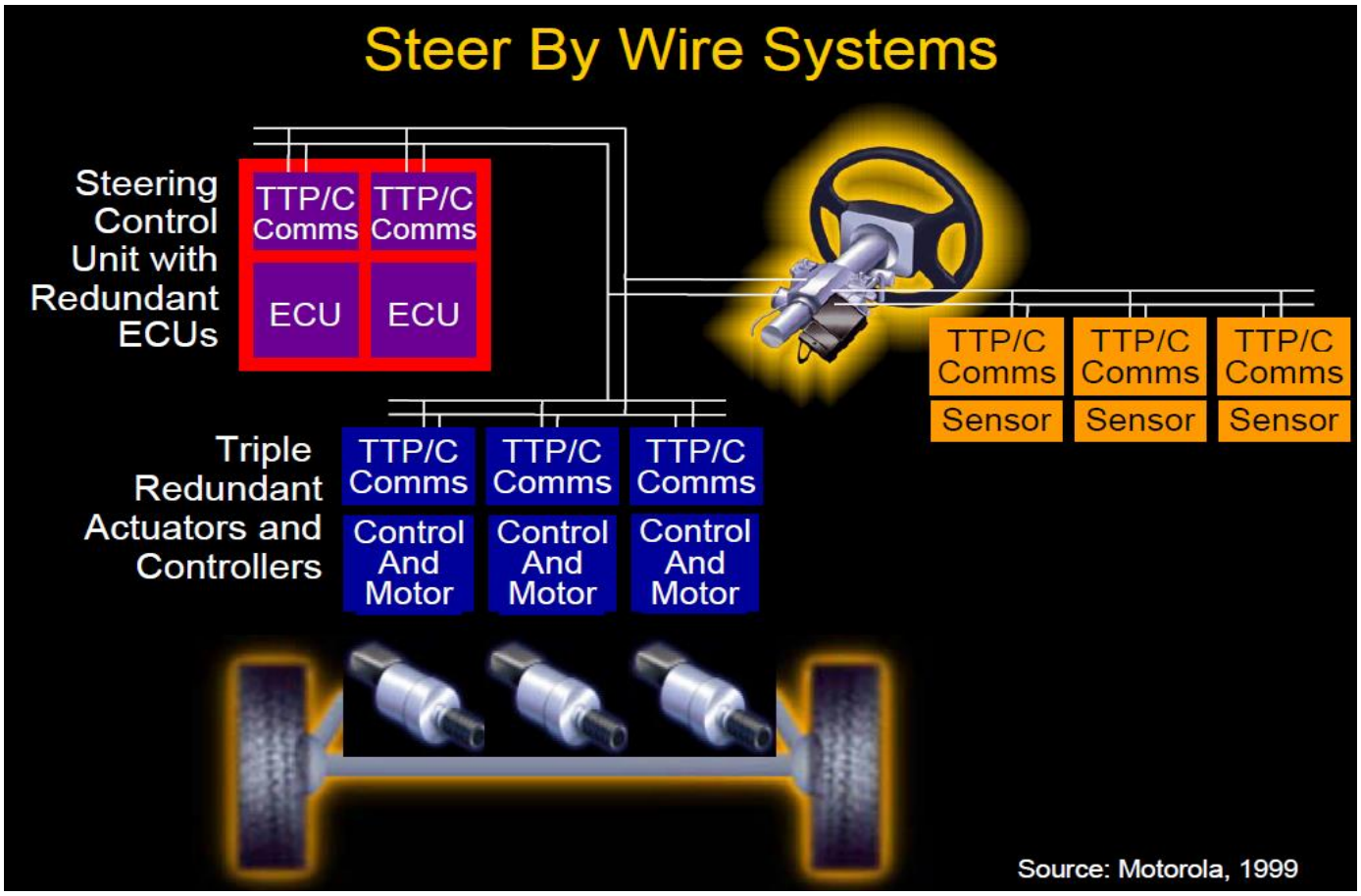
CAN (Controler Area Network) – protokół komunikacyjny o akcjach wyzwalanych zdarzeniami (event-triggered).

TTP (Time -Triggered Protocol) – protokół komunikacyjny o akcjach wyzwalanych czasem (time-triggered).



Do zarządzania urządzeniami wymagającymi niezawodnej obsługi w czasie rzeczywistym wykorzystywany jest protokół TTP.

Sterowniki klasy TT (Time-Triggered)



W wielu przypadkach występują klastry sterująco-monitorujące złożone z kilku zdublowanych modułów SRU (ECU) (SRU - Smallest Replaceable Unit, ECU - Electronic Control Unit).

Klastry TTA

Każdy klaster składa się z kilku węzłów w postaci modułów elektronicznych *SRU/ECU*, które z punktu widzenia systemu komunikacji stanowią najmniejszy, zastępowalny w przypadku awarii element.

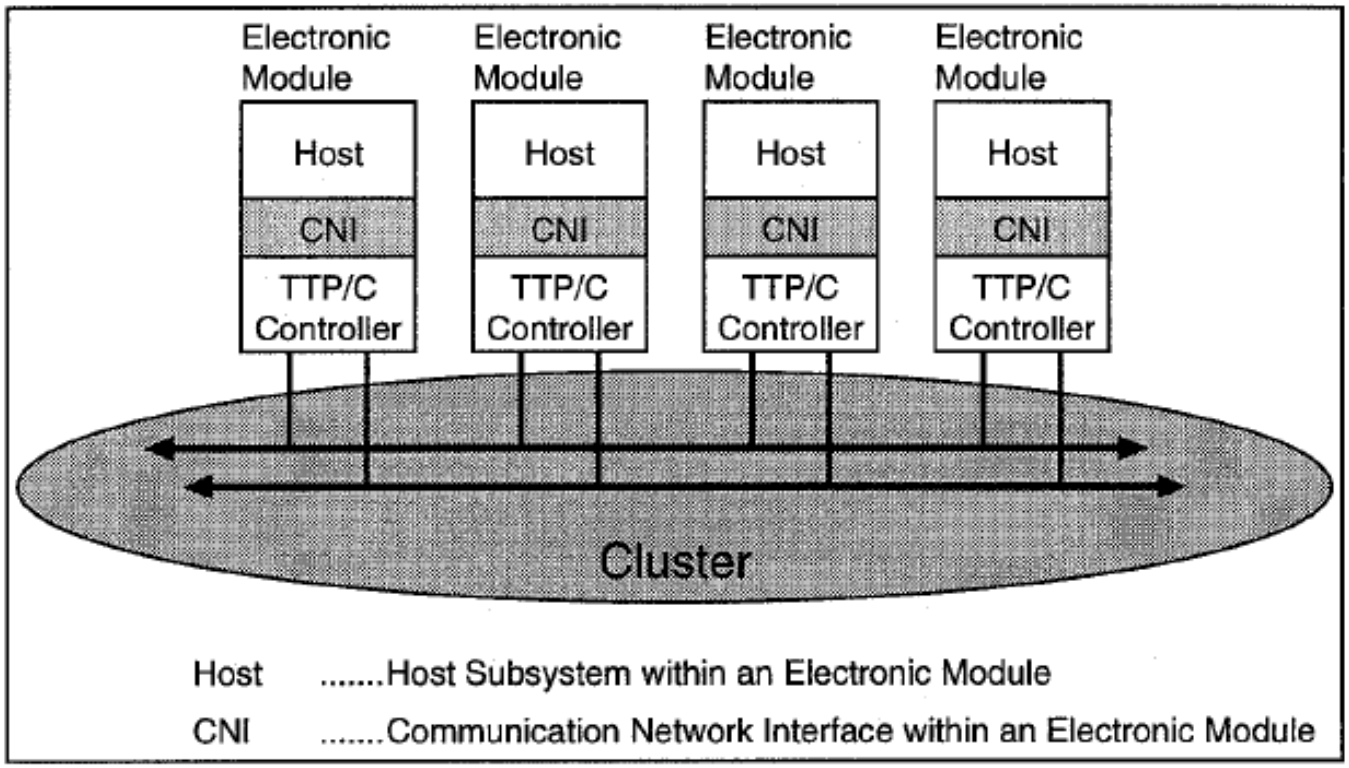
Moduły SRU komunikują się ze sobą z wykorzystaniem wspólnej szyny danych (może być kilka magistrali komunikacyjnych – zwiększenie niezawodności).

Magistrala komunikacyjna jest wykorzystywana przez węzły SRU klastra w trybie wzajemnego wykluczania, a dostępem do niej zarządza protokół *TTP* (Time-Triggered Protocol).

Każdy moduł SRU składa się z komputera (procesora hosta), który wykonuje określone aplikacje (procesy) oraz autonomicznego sterownika komunikacyjnego TTP, który zarządza przesyłaniem wiadomości pomiędzy modułami SRU w oparciu o protokół TTP, np. TTP/C.

Klastry TTA

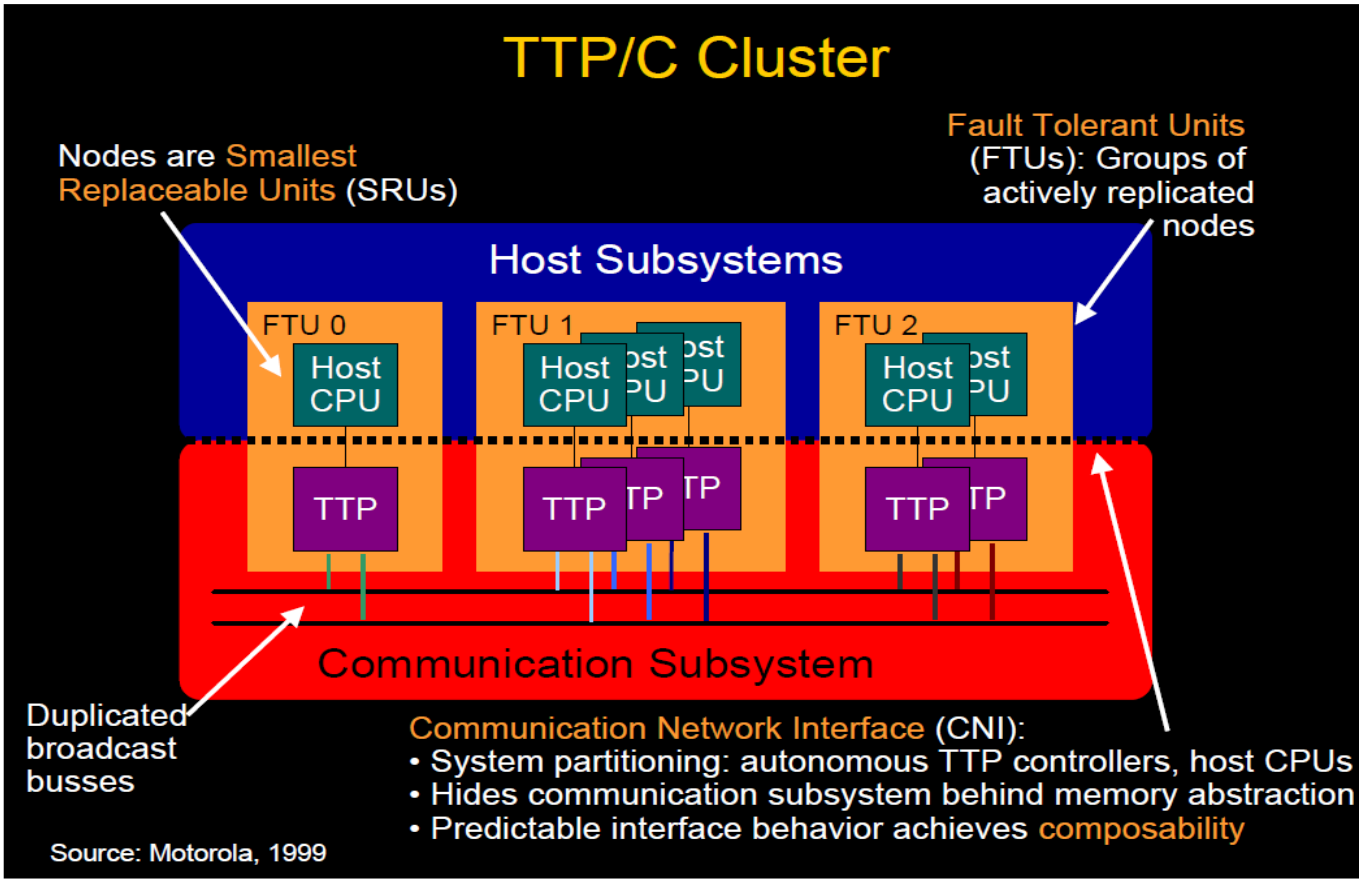
Klaster z czterema pojedynczymi węzłami SRU.



Klastry TTA

W celu zwiększenia niezawodności systemu moduły *SRU* mogą być łączone w grupy funkcjonalne *FTU*, tolerujące uszkodzenia (Fault Tolerant Unit).

Klaster ze zdublowanymi węzłami funkcjonalnymi SRU w postaci modułów FTU.



FTU 0 – jeden moduł SRU;

FTU 1 – trzy moduły SRU;

FTU 2 – dwa moduły SRU;

Każdy moduł SRU ma swój przedział czasowy do wysyłania wiadomości na każdą magistralę w ramach przedziału czasowego przyznanego grupie FTU, do której należy.

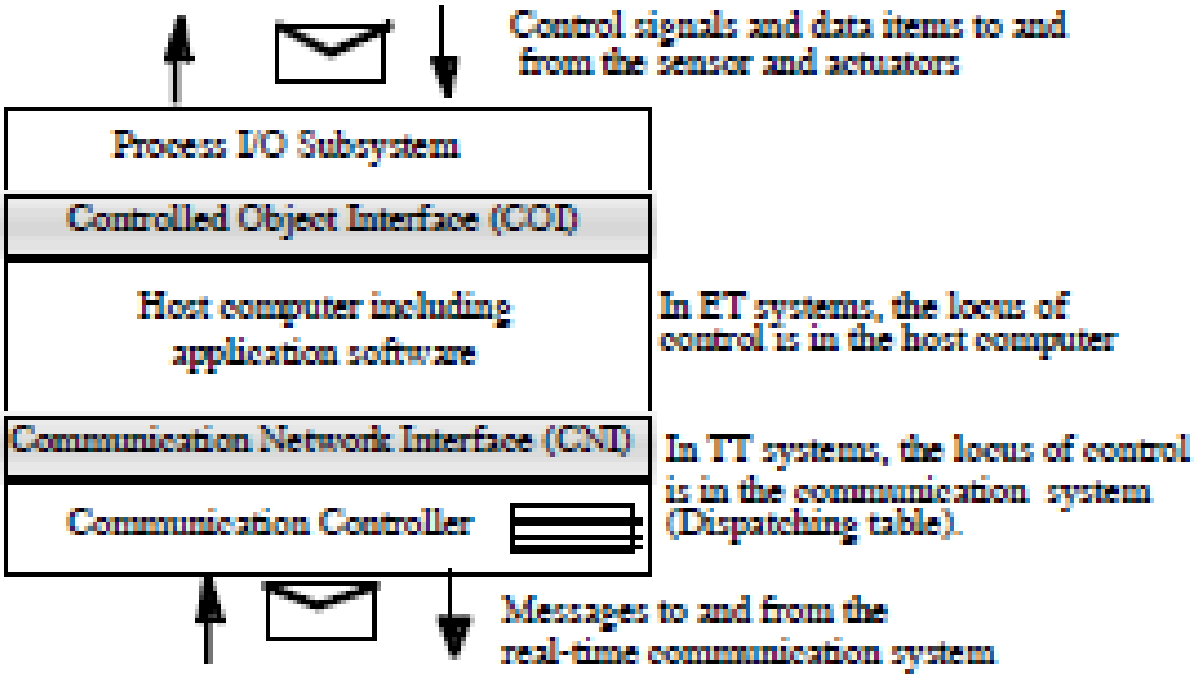
Struktura modułu SRU w klastrze TTA

Główne elementy modułu SRU:

- *komputer sterujący (host computer);*
- *kontroler komunikacyjny (communication controller - CC);*
- *podsystem obsługi procesów wejścia/wyjścia (process I/O subsystem).*

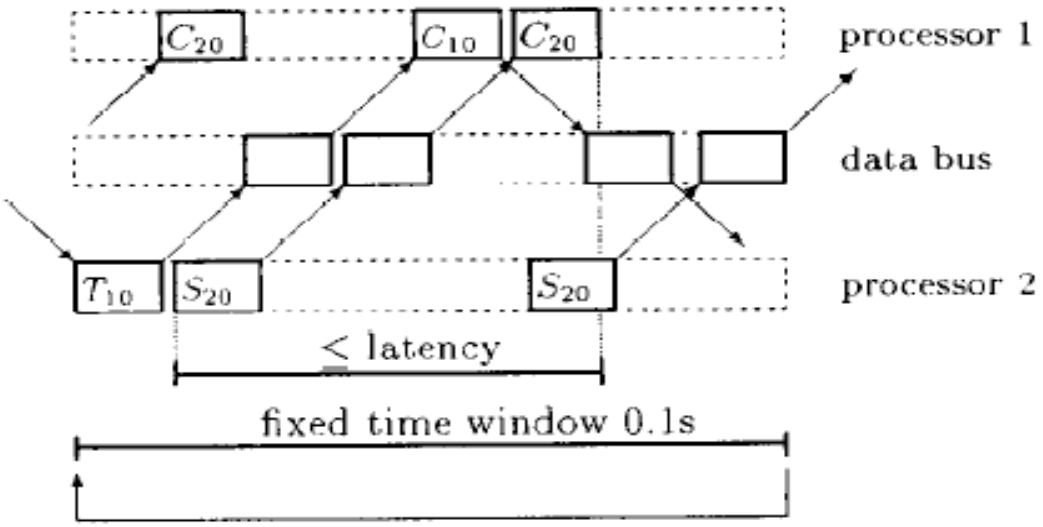
Te trzy podsystemy są połączone przez dwa interfejsy:

- *sieciowy interfejs komunikacyjny (Communication Network Interface CNI);*
- *interfejs zarządzania obiektami wejścia/wyjścia (Controlled Object Interface – COI).*



Komputer modułu SRU w klastrze TTA (*Host komputer)*

- zawiera CPU, pamięć, lokalny zegar czasu rzeczywistego, system operacyjny oraz oprogramowanie aplikacyjne;
- pobiera dane wejściowe poprzez COI oraz CNI oraz dostarcza dane wyjściowe do COI oraz CNI;
- cel: realizacja aplikacji w czasie rzeczywistym z uwzględnieniem ograniczeń czasowych; jeden procesor może obsługiwać w trybie wzajemnego wykluczania kilka procesów współbieżnych (np. procesor 2: T_{10}, S_{20}).



System komunikacyjny w klastrze TTA – opis działania

- system komunikacyjny składa się z kontrolerów CC wszystkich węzłów (modułów SRU), magistrali komunikacyjnych oraz protokołu TTP, zarządzającego dostępem do magistrali;

- działanie kontrolera komunikacyjnego CC jest wyzwalane czasem;

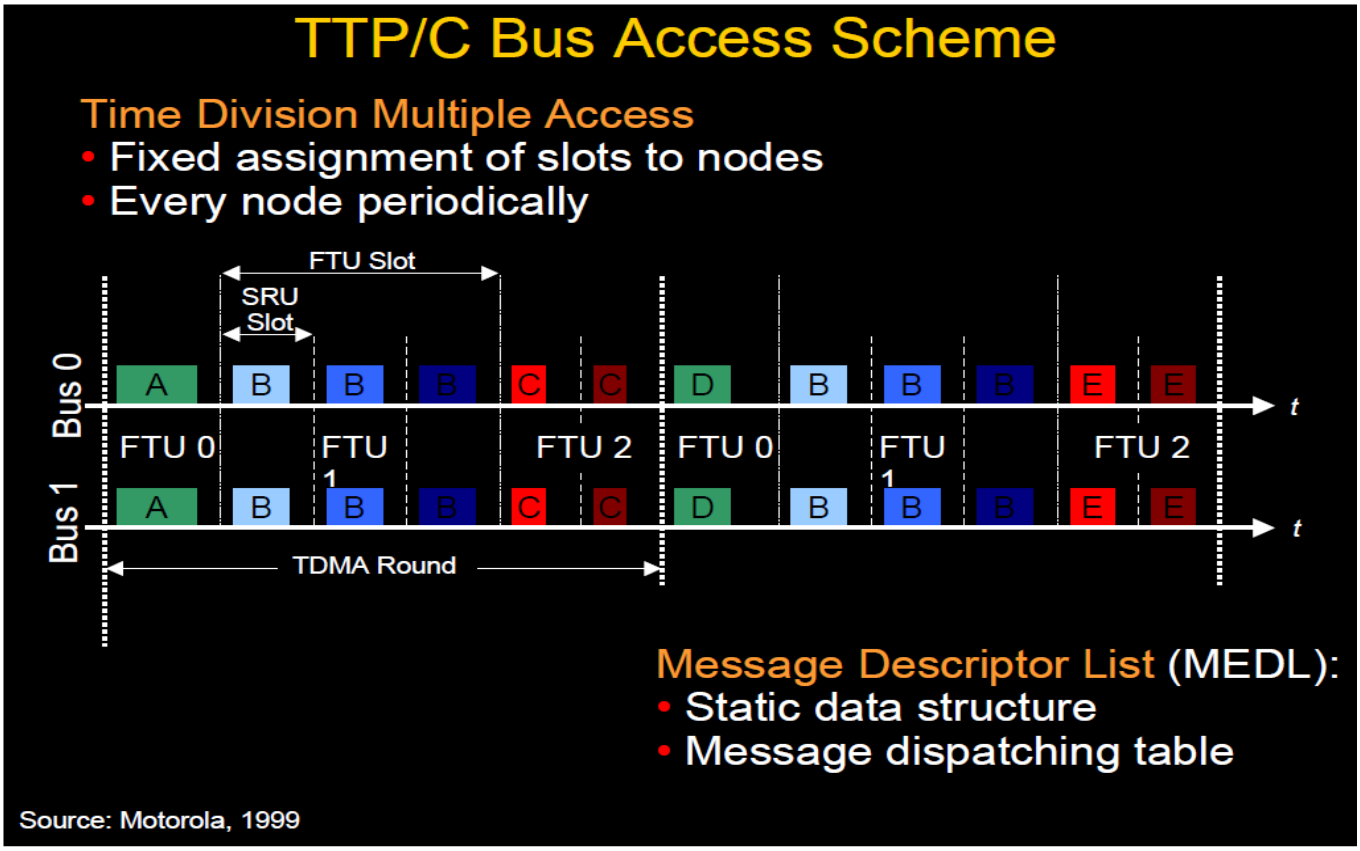
- każdy kontroler CC zawiera w swojej pamięci *globalny harmonogram cykliczny*, w postaci listy MEDL (Message Descriptor List), określający chwile czasowe, w których określona wiadomość ma być wysłana lub odebrana (lokalne zegary modułów SRU są zgodne z czasem globalnym, gdyż są odpowiednio synchronizowane – protokół FTA, Fault Tolerant Averaging Algorithm);

- dostęp do magistrali jest realizowany w oparciu o protokół TTP, który wykorzystuje cykliczną metodę dostępu do łącza TDMA (Time-Division Multiple Access), która zapewnia bezkonfliktowy dostęp do magistrali poszczególnym modułom SRU klastra;

- każdy moduł ma przypisany unikalny przedział czasowy (slot) w ramach jednej rundy TDMA, w którym może wysyłać swoje ramki z danymi (moduł SRU, który chce nadawać musi czekać na swój slot; inne mogą w tym czasie odbierać lub wykonywać aplikacje);

- slot modułu FTU składa się ze slotów modułów SRU, które są w nim zawarte;

- po zakończeniu jednej rundy TDMA wykonywane są kolejne rundy, o takiej samej strukturze slotów, ale innych wiadomościach (np. najpierw wiadomości (A, B, B, B, C, C), potem (D, B, B, B, E, E)); liczba różnych rund TDMA określa długość *czasu cyklu* klastra TTA (okres powtarzania cyklu komunikacyjnego).

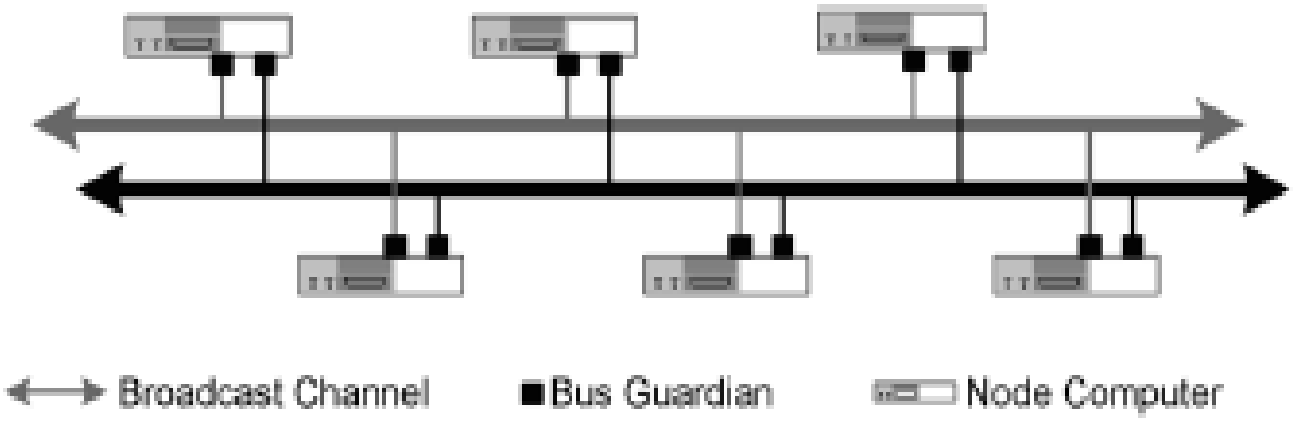


Harmonogramowanie cykliczne w systemach TTA

- działanie systemu komunikacyjnego klastra opiera się na cyklicznym powtarzaniu harmonogramu zapisanego w tabeli MEDL, która jest skopiowana w każdym węźle klastra;

- każdy kontroler CC korzysta z dwóch równorzędnych magistrali komunikacyjnych, na których powielane są identyczne wiadomości;

- komunikacja na magistralach jest obsługiwana przez protokół TTP.



Struktura listy MEDL (harmonogram cykliczny klastra)

Lista MEDL (Message Descriptor List), składowana w każdym kontrolerze CC, zawiera informacje o tym kiedy (zgodnie z globalnym zegarem) określona wiadomość musi być wysłana lub odebrana z podsystemu komunikacyjnego, a także pozycję danych w interfejsie komunikacyjnym CNI.

Długość listy jest równa długości *czasu cyklu klastra* i składa się z sekwencji rund TDMA, które są następnie powtarzane cyklicznie.

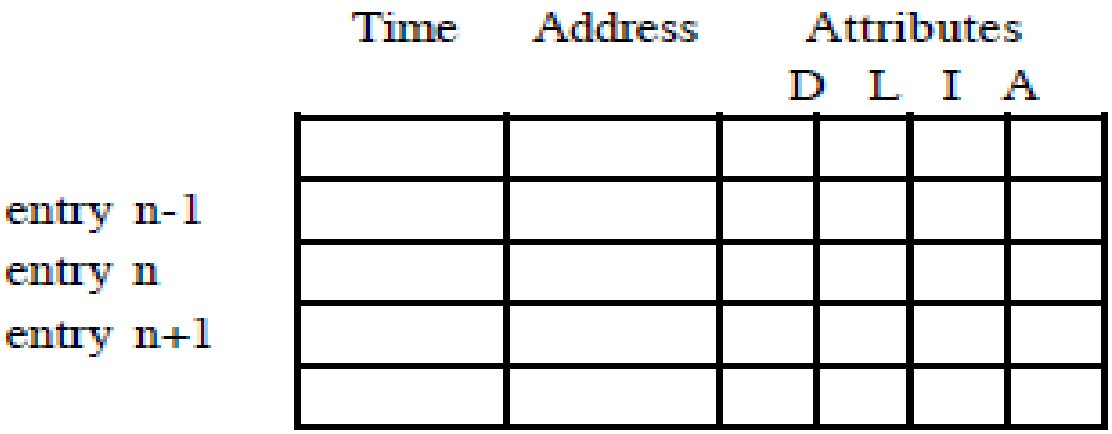
Każdy wpis w tabeli MEDL składa się z trzech pól:

- ***pola czasu***; zawiera czas globalny, określający kiedy wiadomość ma być wysłana lub odebrana przez każdy z modułów SRU; w danej chwili tylko jeden moduł SRU może wysyłać dane – inne odbierają; przy czym jedna z aplikacji modułu może wysyłać dane poprzez kontroler CC, a inna wykonywać zadania z wykorzystaniem procesora hosta;

- ***pola adresu***; wskazuje na obiekty, w których dane mają być składowane (odbieranie), lub z których mają być pobierane (nadawanie);

- ***pola atrybutów***; zawiera informacje o typie i parametrach wiadomości (wejściowa/wyjściowa, długość, N/I, parametry dodatkowe).

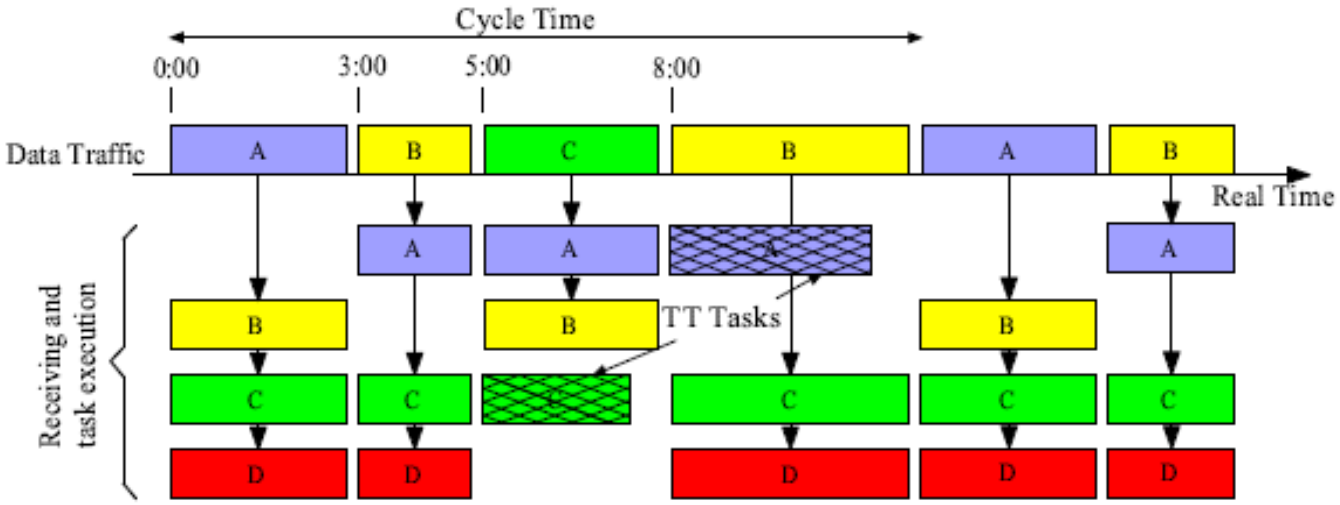
Struktura listy MEDL składowanej w kontrolerze CC/TTP



Przykładowy harmonogram cykliczny klastra

Time	[0:00,3:00)	[3:00,5:00)	[5:00,8:00)	[8:00, 12:00)
Node A	send	receive	receive	execute task
Node B	receive	send	receive	send
Node C	receive	receive	execute task	receive
Node D	receive	receive	idle	receive

Table 1: Example for a time-triggered schedule

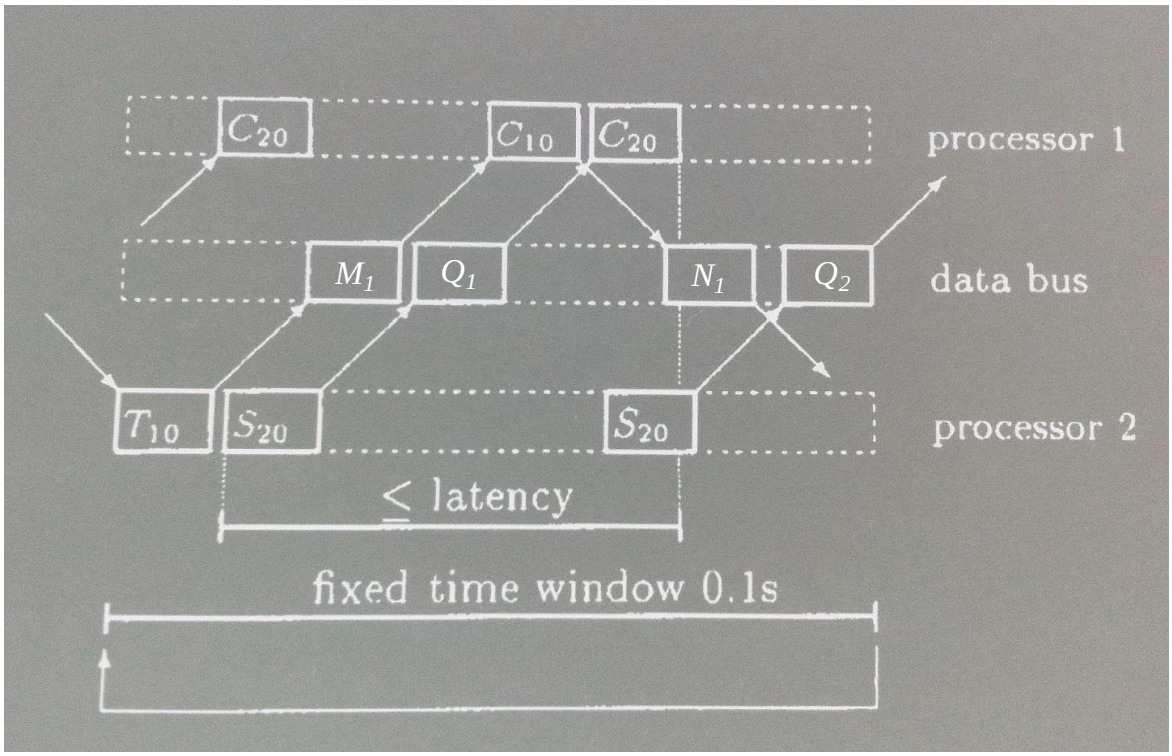


Problem harmonogramowania cyklicznego w systemach TTA

- Współdzielenie magistrali w trybie wzajemnego wykluczania oznacza, że operacje przesyłania wiadomości przez procesy muszą być „serializowane” na magistrali jedna za drugą w sposób bezkonfliktowy.

- W wielu przypadkach, na operacje realizowane w systemach dynamicznych czasu rzeczywistego nakładane są *„twarde”* ograniczenia na czasy przetwarzania danych lub przesyłania wiadomości od procesu nadawcy do procesu odbiorcy (np. czas, który może upłynąć od momentu wysłania danej przez czujnik temperatury do momentu jej odebrania i przetworzenia przez sterownik wykrywający uszkodzenia, np. silnika, układu hamulcowego, nie powinien przekroczyć dopuszczalnego ograniczenia czasowego - *latency*).

Opis działania przykładowego systemu TTA



- Np. rozpatrzmy proces obsługi termostatuT_{10} wysyłający z częstotliwością 10Hz dane o aktualnej temperaturze do kontrolera C_{10} urządzenia (np. silnika);
- niech ten kontroler, z taką samą częstotliwością, przesyła informacje o potencjalnej awarii do innego kontrolera;
- procesy związane z działaniem T_{10} oraz C_{10} są obsługiwane przez różne procesory, które korzystają z szyny danych współdzielonej w trybie wzajemnego wykluczania;
- każde przesłanie danych związane jest z umieszczeniem określonej wiadomości M_1, N_1, Q_1, Q_2 na szynie danych (konieczność szeregowania wiadomości jedna za drugą ze względu na wzajemne wykluczanie procesów w dostępie do wspólnej szyny);
- dodatkowo załóżmy, że w systemie jest drugi kontroler C_{20}, który cyklicznie, z częstotliwością 20Hz, odbiera aktualne dane z sensora S_{20};
- kontrolery C_{10}, C_{20} są obsługiwane cyklicznie, w trybie wzajemnego wykluczania, przez procesor 1, natomiast czujniki T_{10}, S_{20} przez procesor 2;
- istnieją ograniczenia czasowe na sposób działania procesów, np. *przetworzona wiadomość nie musi być wysyłana od razu po zakończeniu procesu nadawczego*; odległość czasowa pomiędzy chwilą wysłania wiadomości przez proces nadawczy S_{20}, a jej przetworzeniem przez proces odbiorczy C_{20} nie może przekroczyć zadanego ograniczenia (*latency*).

Problem szeregowania

Wyznaczyć bezkonfliktowy harmonogram cykliczny o ustalonym czasie cyklu *CT* (tj. czasy rozpoczęcia poszczególnych operacji w systemie), który odzwierciedla prawidłowe działanie systemu procesów, tzn. spełnia zadane ograniczenia dotyczące:

- czasów wykonywania i powtarzania procesów: np.
  $compl(P_i) = start(P_i) + dur(P)$; $start(P_i) = start(P_{i-1}) + period(P)$

- czasów transferu wiadomości: np.
  $compl(M_i) = start(M_i) + dur(M)$; $start(M_i) = start(M_{i-1}) + period$

- w ramach cyklu CT czasy liczone od [0, …, CT];

compl – zakończenie; start – rozpoczęcie; dur – czas trwania;

- relacji czasowych pomiędzy procesami: np.

$compl(S_i) <= start(M_i)$;
wiadomość M_i wysyłana po zakończeniu procesu nadawczego S_i;

$compl(M_i) <= start(S_{i+1})$;
wiadomość M_i musi być dostarczona przed kolejnym wykonaniem nadawcy S_{i+1}

$compl(P_i) <= CT$;

$compl(M_i) <= CT$.

Jeśli $start(R_i) >= compl(M_i)$, to $compl(R_i) - start(S_i) <= latency(M, R)$;

gdzie S_i – kolejne wykonanie procesu nadawczego;
R_i – kolejne wykonanie procesu odbiorczego;

Przesyłanie wiadomości przez wspólną szynę danych, między różnymi procesorami, uwzględnia wzajemne wykluczanie operacji transmisji danych.

Stąd, czasy rozpoczęcia i zakończenia wiadomości M_i, N_j spełniają zależności:

$compl(M_i) <= start(N_j)$ or $compl(N_j) <= start(M_i)$, gdzie $M \neq N$.

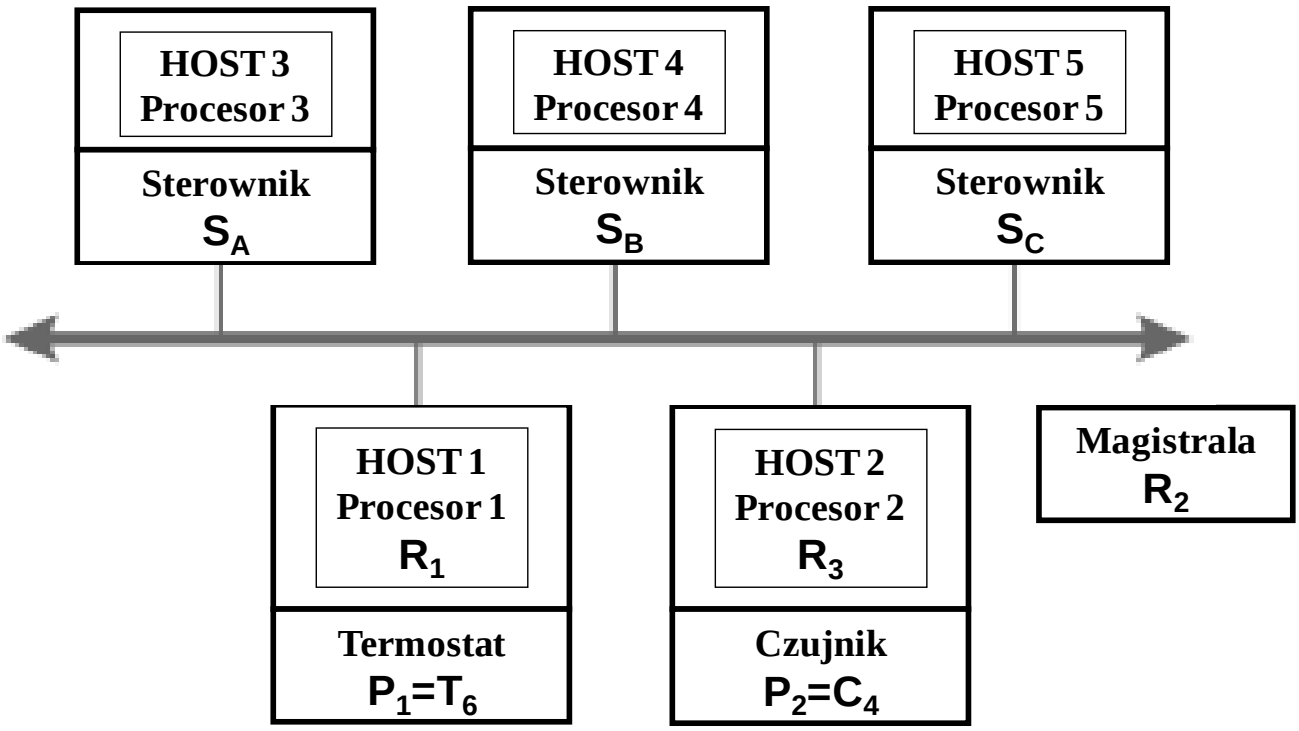
Uwzględniane mogą być też inne ograniczenia.

Istnienie rozwiązań

Badania symulacyjne układu złożonego z *n procesów cyklicznych* (tzw. *n-proces*), które współdzielą magistralę w trybie wzajemnego wykluczania pokazują, że dla systemu procesów o ustalonej strukturze (zadanych czasach operacji, czasach cyklu, ograniczeniach na czasy przetwarzania danych i przesyłania wiadomości) nie zawsze istnieje dopuszczalny, tj. spełniający te ograniczenia harmonogram cykliczny.

Schemat systemu TTA z pięcioma węzłami

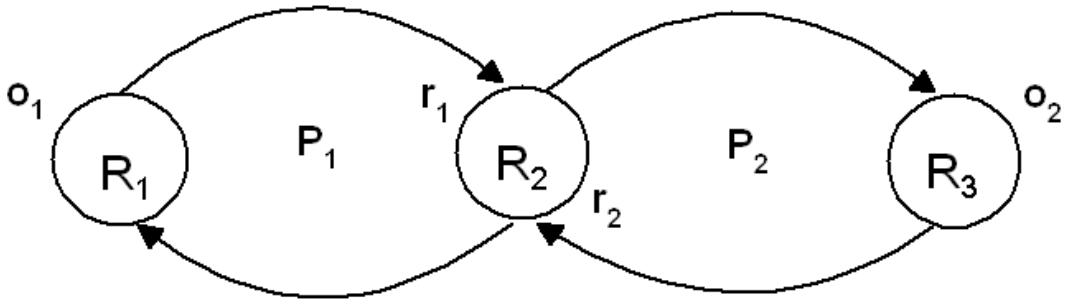
Proces P_1 (T_6) wysyła cyklicznie co 6 u.j.c. (ujedn. jedn. czasu) stan termostatu. Proces P_2 (C_4) wysyła cyklicznie co 4 u.j.c. stan czujnika.

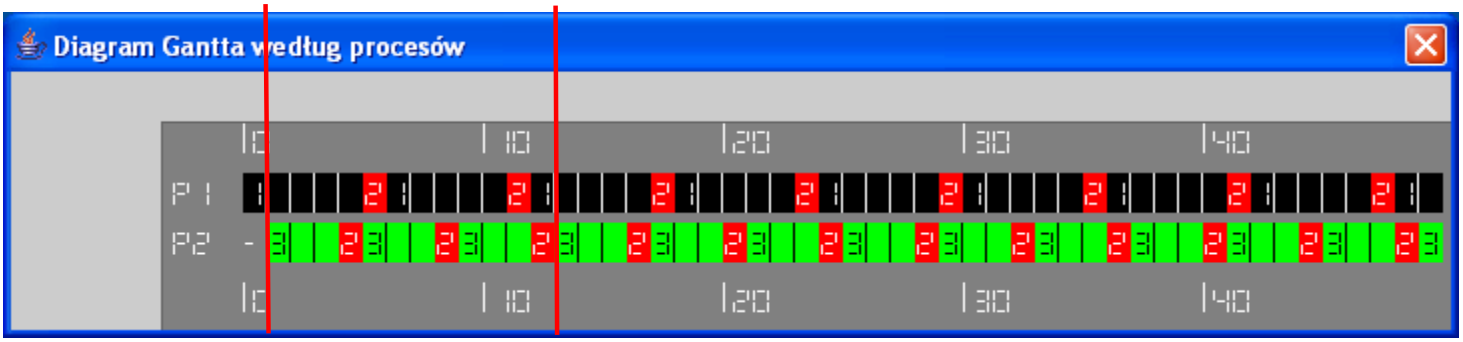


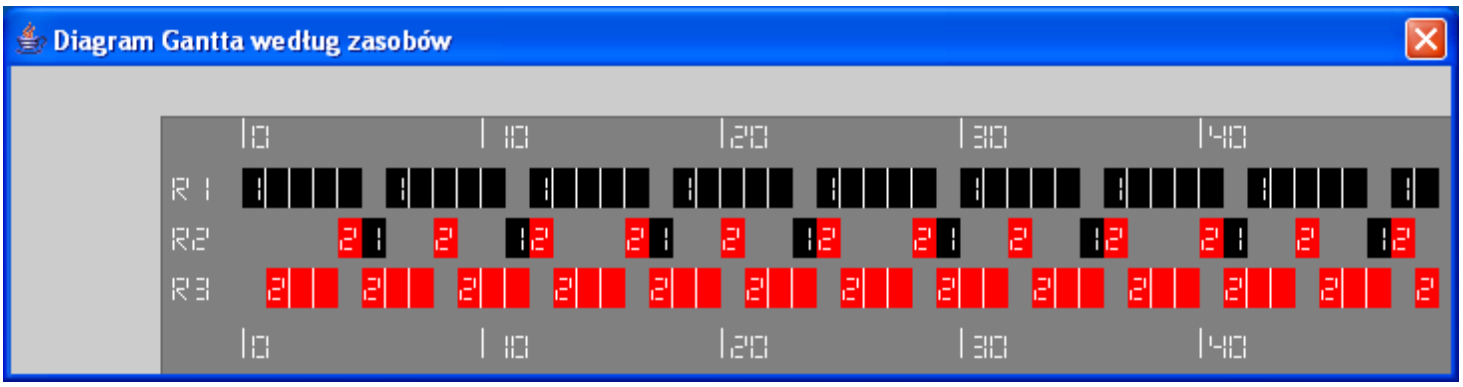
Harmonogram systemu TTA

Prawidłowy harmonogram działania systemu powinien zapewniać bezkonfliktowy dostęp procesów P_1 (T_6) – czas cyklu 6 i P_2 (C_4) – czas cyklu 4, wysyłających ramki z danymi do sterowników S_A, S_B, S_C, do wspólnej magistrali R_2, tj. nie powinno występować oczekiwanie procesów P_1 i P_2 na dostęp do zasobu R_2.

Przykład 1. System TTA, dla którego istnieje harmonogram bezkonfliktowy. Czas cyklu takiego harmonogramu jest równy NWW (Najmniejsza Wspólna Wielokrotność) czasów cykli procesów składowych.







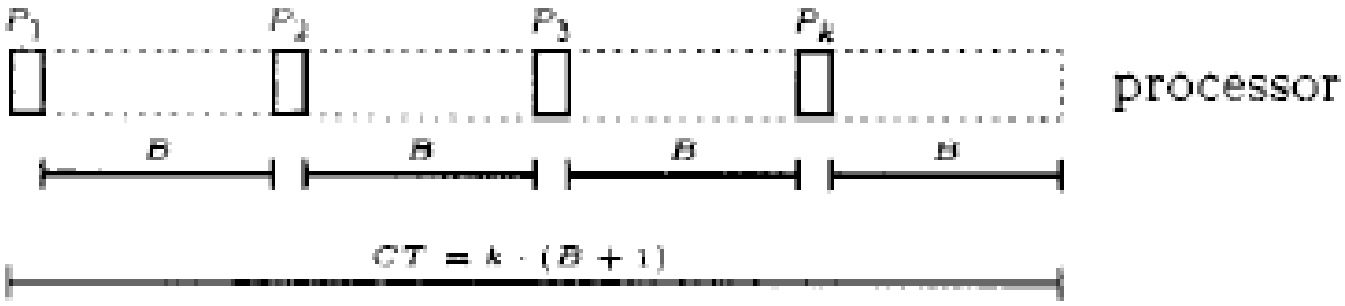
W przypadku niewłaściwie dobranych czasów operacji (przetwarzania i przesyłania danych) oraz czasów cyklu procesów (P_1 - czas (T_7) i P_2 - czas (C_5)) bezkonfliktowy harmonogram cykliczny może nie istnieć.

Przykład 2. System TTA, dla którego niezależnie od stanu początkowego pojawia się przebieg cykliczny z konfliktami na zasobie wspólnym R_2 (procesy P_1 i P_2 oczekują na zasób wspólny R_2).



Złożoność obliczeniowa problemu decyzyjnego szeregowania TTA

Problem decyzyjny znajdowania harmonogramu cyklicznego dla jednego zasobu i n procesów współdzielących ten zasób jest NP-zupełny, gdyż jego szczególnym przypadkiem jest decyzyjny problem pakowania – *bin packing problem;* np. problem rozłożenia n-1 procesów cyklicznych o okresie CT = k(B+1) i czasach dostępu do wspólnego zasobu u_a, (a=1,...,n) w k oknach czasowych – przedziałach (wersja optymalizacyjna NP-trudna, gdy szukamy min k) o długości B wyznaczonych przez proces P, o czasie cyklu (B+1), i czasie dostępu do zasobu wspólnego 1. Czas cyklu układu wynosi NWW(B+1, k(B+1)) = k(B+1).



[ ] Schild K., Wurtz J., *Scheduling of Time-Triggered Real-Time Systems*, Constraints, 5, 335-357, Kluwer AP, Boston, 2000.

2.2. Zarządzanie procesami i zasobami

Ze względów pojęciowych lub projektowych *fragmenty kodu jądra systemu operacyjnego*, związane z obsługą procesów i zasobów, wyodrębnia się postaci zarządców.

Zarządca procesów (proces manager) — kontroluje stany procesów w celu efektywnego i bezpiecznego wykorzystania współdzielonych zasobów systemu; zarządca grupuje i wykonuje funkcje obsługi procesów.

Zarządca zasobów (resource manager) — realizuje przydział zasobów stosownie do żądań procesów, aktualnego stanu systemu oraz ogólnosystemowej polityki przydziału; kontroluje stan zajętości zasobów.

Struktury danych wykorzystywane do opisu stanu procesów i zasobów

Zarządzenie wymaga odpowiednich struktur danych, w których przechowywane są informacje na potrzeby ewidencji stanu procesów i zasobów, ich powiązań, potrzeb zasobowych procesów, itp.

Deskryptor procesu (blok kontrolny procesu, PCB) — używany przez zarządcę procesów w celu rejestrowania stanu procesu w czasie jego monitorowania i kontroli.

PCB zawiera niezbędne informacje, umożliwiające odpowiednie zarządzanie procesem. Struktura bloku może być różna w zależności od celów projektowych systemu operacyjnego lub przyjętych rozwiązań implementacyjnych.

Na ogół do zarządzania procesem niezbędne są następujące informacje:

- Stan procesu, potrzebny do podjęcia decyzji odnośnie dalszego losu procesu (np. usunięcia procesu i zwolnienia zasobów, przesunięcia procesu z pamięci fizycznej do pamięci pomocniczej (dyskowej) lub odwrotnie itp.).

- Licznik rozkazów i stan rejestrów; niezbędne do odtworzenia kontekstu danego procesu (odtworzenie stanu po przełączeniu procesora na inny proces).

- Informacje niezbędne do planowania przydziału procesora do procesów, które umożliwiają właściwe szeregowanie procesów i podejmowanie decyzji przez planistów (ang. scheduler); część tych informacji może znajdować się poza właściwym deskryptorem procesu;

- Informacje o zajętości pamięci; umożliwiają ochronę obszarów pamięci, w szczególności powstrzymanie procesu przed ingerencją w obszary poza jego przestrzenią adresową.

- Informacje o stanie wejścia-wyjścia, obejmujące dane o przydzielonych urządzeniach, wykaz otwartych plików, itp.; umożliwiają one odpowiednie zarządzanie tymi zasobami i dostępem do nich przez system operacyjny, a także ich odzyskiwanie po zakończeniu procesu itp.

Deskryptor zasobu — przechowuje informacje o dostępności i zajętości danego typu zasobu; w zależności od rodzaju zasobu struktura opisu może być bardzo różna, narzucona przez rozwiązanie przyjęte na poziomie architektury procesora (np. w przypadku pamięci), a czasami wynika z decyzji projektowych; każdy zasób może składać się z kilku jednostek (np. może być kilka buforów pamięci) – w deskryptorze zasobu powinna być zapisana informacja o liczbie jednostek zasobu.

Rodzaje zasobów

Podział ze względu na sposób wykorzystania:
– zasoby odzyskiwalne (zwrotne, ang. reusable),
– zasoby nieodzyskiwalne (niezwrotne, zużywalne, ang. consumable).

Podział ze względu na sposób odzyskiwania:
– zasoby wywłaszczalne,
– zasoby niewywłaszczalne.

Podział ze względu na tryb dostępu:
– współdzielone,
– wyłączne.

Zasoby mogą być odzyskane po zakończeniu procesów, np. pamięć, bufory urządzeń. Niektóre zasoby wykorzystywane przez proces nie są odzyskiwane do systemu. Proces zużywa takie zasoby w ramach przetwarzania lub wytwarza nowe zasoby. Na przykład proces zużywa energię, zużywa czas procesora przed linią krytyczną (terminowym czasem zakończenia procesu), istotny w systemach czasu rzeczywistego, a wytwarza dane, sygnały synchronizujące, które nie są odzyskiwane do systemu po zakończeniu procesu.

Jeśli zasób można odzyskać, to istotny z punktu widzenia pewnych problemów, (np. zakleszczenia procesów) może być sposób odzyskiwania. Zasób wywłaszczalny można odebrać procesowi (np. procesor). Natomiast jednostki zasobu niewywłaszczalnego proces sam musi zwrócić do systemu. Z punktu widzenia systemu oznacza to, że należy poczekać, aż proces, posiadający zasób, dojdzie do takiego stanu przetwarzania, w którym zasób nie będzie mu już potrzebny (np. po wydrukowaniu danych proces zwróci drukarkę, ale nie papier ani toner).

Pewne zasoby mogą być używane współbieżnie przez wiele procesów, np. segment kodu programu może być czytany i wykonywany przez wiele procesów w tym samym czasie. Są też zasoby dostępne w trybie wyłącznym, czyli dostępne, co najwyżej dla jednego procesu w danej chwili czasu (np. drukarka, deskryptor procesu w tablicy procesów).

Stany procesu

W zależności od stanu wykonywania programu i dostępności zasobów można wyróżnić następujące, ogólne stany procesu w systemie współbieżnym: nowy, gotowy, wykonywany, oczekujący, zakończony.



Nowy — formowanie procesu, czyli gromadzenie zasobów niezbędnych do rozpoczęcia wykonywania procesu (np. pamięci fizycznej), z wyjątkiem procesora (kwantu czasu procesora), a po zakończeniu formowania oczekiwanie na przyjęcie do kolejki procesów gotowych.

Gotowy — oczekiwanie na przydział kwantu czasu procesora (dostępność wszystkich niezbędnych zasobów z wyjątkiem procesora).

Wykonywany — wykonywanie instrukcji programu danego procesu; dokonywanie zmian stanu odpowiednich zasobów systemu w związku z realizacją określonego programu.

Oczekujący — zatrzymanie wykonywania instrukcji programu danego procesu ze względu na potrzebę przydziału dodatkowych zasobów, konieczność otrzymania danych od innego procesu, zakończenia działania innego procesu lub osiągnięcia odpowiedniego stanu przez otoczenie zewnętrzne procesu (np. zakończenie operacji wejścia/wyjścia przez urządzenie zewnętrzne); mogą być w tym stanie procesy, np. znajdujące się w kolejce procesów oczekujących na dostęp do urządzenia wejścia/wyjścia; po uzyskaniu dostępu proces przechodzi do kolejki procesów gotowych; każde urządzenie ma swoją kolejkę).

Zakończony — zakończenie wykonywania programu, zwolnienie większości zasobów i oczekiwanie na możliwość przekazania informacji o zakończeniu innym procesom lub jądru systemu operacyjnego.

Przejście ze stanu *gotowy* do *wykonywany* wynika z decyzji modułu szeregującego procesy (planisty przydziału procesora), która oparta jest na priorytetach procesów. Jeśli w systemie jest jedna jednostka przetwarzająca (procesor), to w stanie wykonywany może być tylko jeden proces, podczas gdy pozostałe procesy znajdują się w innych stanach (w szczególności w stanie gotowy).

Przejście ze stanu *wykonywany* bezpośrednio do stanu *gotowy* oznacza wywłaszczenie procesu z procesora. Wywłaszczenie może być następstwem:
- upływu kwantu czasu w systemach z podziałem czasu,
- pojawienia się procesu gotowego z wyższym priorytetem w systemie z priorytetami dynamicznymi (np. przerwanie procesu pochodzącego z kolejki priorytetowej o niższym priorytecie).

W *systemie z podziałem czasu* proces otrzymuje tylko kwant czasu na wykonanie kolejnych instrukcji. Upływ kwantu czasu odmierzany jest przez przerwanie zegarowe, a po stwierdzeniu wyczerpania kwantu czasu następuje przełączenie kontekstu i kolejny kwant czasu otrzymuje inny proces (rotacyjny algorytm planowania przydziału procesora).

W *systemie z dynamicznymi priorytetami* przerwanie zegarowe lub inne zdarzenie obsługiwane przez jądro wyznacza momenty czasu, w których przeliczane są priorytety procesów. Jeśli stosowane jest wywłaszczeniowe podejście do planowania przydziału procesora, oparte na priorytecie, wówczas proces o najwyższym priorytecie otrzymuje procesor.

Stan procesu zależy od dostępności zasobów systemu; w zależności od tego, jakie zasoby są dostępne, proces jest albo wykonywany, albo na coś czeka (w stanach NOWY, GOTOWY, OCZEKUJĄCY). W czasie oczekiwania proces trafia do kolejki (zbioru) procesów oczekujących na dany zasób. Opuszczając jedną kolejkę proces często trafia do innej, np. opuszczając kolejkę do urządzenia, trafia do kolejki procesów gotowych.

Rodzaje kolejek w systemie operacyjnym:

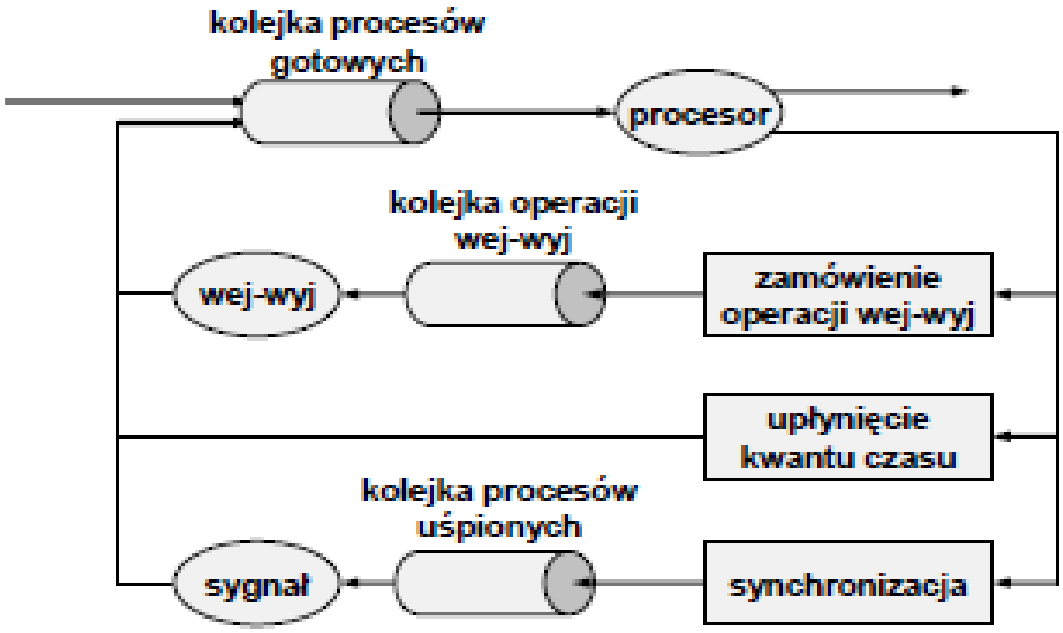
Kolejka zadań (ang. job queue) — wszystkie procesy systemu.

Kolejka procesów gotowych (ang. ready queue) — procesy gotowe do działania, przebywające w pamięci głównej.

Kolejka do urządzenia (ang. device queue) — procesy czekające na zakończenie operacji wejścia-wyjścia na danym urządzeniu (może być kilka takich kolejek w zależności od liczby urządzeń).

Kolejka procesów oczekujących na sygnał synchronizacji od innych procesów (np. kolejka procesów oczekujących na semaforze).

Diagram kolejek w planowaniu przydziału procesora



Koniec wykonywania

Po wyjściu ze stanu wykonywany (po opuszczeniu procesora), w zależności od przyczyny zaprzestania wykonywania programu, proces jest usuwany z pamięci lub trafia do jednej z kolejek (procesów gotowych, oczekujących na obsługę wej./wyj., lub oczekujących na synchronizację).

Koniec kwantu czasu

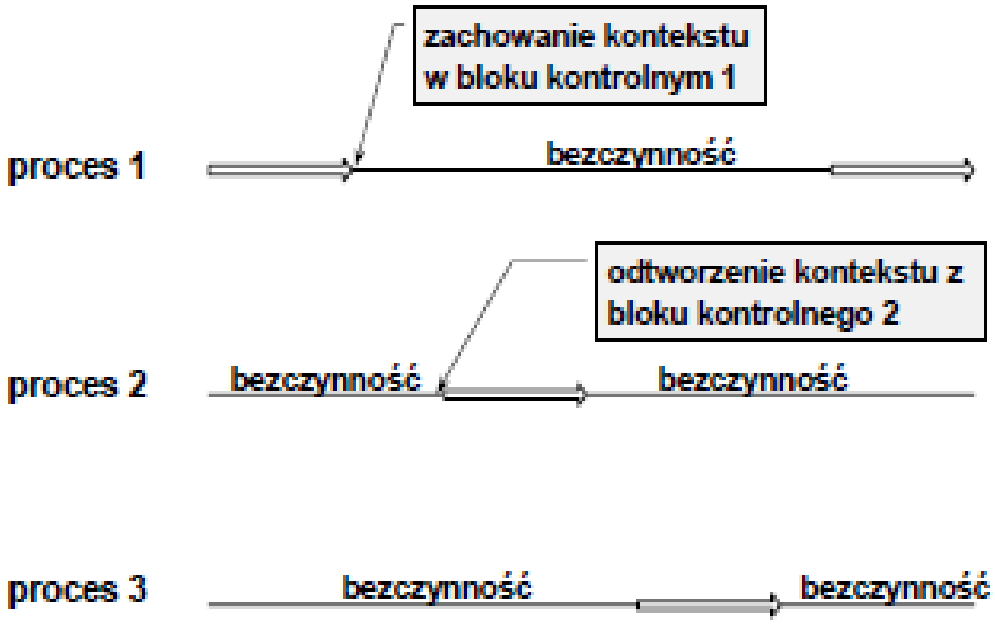
Po upłynięciu kwantu czasu proces (w stanie gotowy) trafia na ***koniec*** kolejki procesów gotowych, a ***z czoła*** tej kolejki pobierany jest proces zaplanowany do wykonywania. Następuje, więc przełączenie procesora w kontekst nowego procesu. Odpowiednio częste przełączanie kontekstu przy niezbyt długim oczekiwaniu w kolejce procesów gotowych umożliwia na bieżąco (ang. on-line) interakcję procesu z użytkownikiem, dzięki czemu użytkownik ma wrażenie, że wyłącznie jego zadania są wykonywane przez system. Tak, więc użytkownik nie ma poczucia dyskomfortu nawet, gdy zasoby systemu są w rzeczywistości współdzielone przez współbieżnie działające procesy wielu użytkowników.

Oczekiwanie na obsługę wejścia/wyjścia lub synchronizację

Jeśli proces opuszcza procesor z innych przyczyn, niż upłynięcie kwantu czasu, przechodzi do stanu oczekujący, co wiąże się z umieszczeniem go w kolejce procesów oczekujących na zajście określonego zdarzenia. Kolejek takich może być wiele, np. kolejka może być związana z każdym urządzeniem zewnętrznym (wej./wyj.), z mechanizmami synchronizacji procesów (oczekiwanie na wiadomość od innego procesu, obsługą przerwania, itp.).

Przełączanie kontekstu

Opuszczenie stanu wykonywany przez jeden proces udostępnia procesor innemu procesowi i następuje wówczas przełączenie kontekstu, które polega na zachowaniu stanu przetwarzania procesu oddającego procesor (zachowaniu kontekstu) i załadowaniu stanu przetwarzania innego procesu (odtworzenie kontekstu).



Szeregowanie procesów

Zarządzanie przechodzeniem procesów z jednej kolejki do innej, a także wybieranie procesów gotowych do uruchomienia na procesorze jest realizowane przez procesy systemowe (w jądrze, lub zwykłe usługi) nazywane **planistami** (programami szeregującymi).

- Planista krótkoterminowy, planista przydziału procesora (ang. CPU scheduler) — zajmuje się przydziałem procesora do procesów gotowych.

- Planista średnioterminowy (ang. medium-term scheduler) — zajmuje się wymianą procesów pomiędzy pamięcią główną a pamięcią zewnętrzną (np. dyskiem).

- Planista długoterminowy, planista zadań (ang. long-term scheduler, job scheduler) — zajmuje się ładowaniem nowych programów do pamięci i kontrolą liczby zadań w systemie oraz ich odpowiednim doborem w celu zrównoważenia wykorzystania zasobów.

Kryteria wyboru (szeregowania) procesów

Zadaniem planistów (programów szeregujących) jest wybieranie procesów z pewnego zbioru tak, aby dążyć do optymalizacji przetwarzania w systemie.

Kryteria optymalizacji mogą być bardzo zróżnicowane.

1) Optymalizacja wykorzystania zasobów poprzez równoważenie obciążenia systemu (procesora, urządzeń zewnętrznych).

2) Minimalizacja czasu odpowiedzi systemu (skrócenie czasu realizacji zadań).

Rola poszczególnych planistów w różnego typu systemach komputerowych może być większa lub mniejsza. W systemach interaktywnych zmniejsza się (lub zupełnie znika) rola planisty długoterminowego, a rośnie rola planisty krótkoterminowego. W systemach wsadowych jest dokładnie odwrotnie.

W wyniku przełączania kontekstu proces oddaje procesor, a stan procesora, związany z danym procesem jest zapisywany w bloku kontrolnym PCB.

Plik wymiany

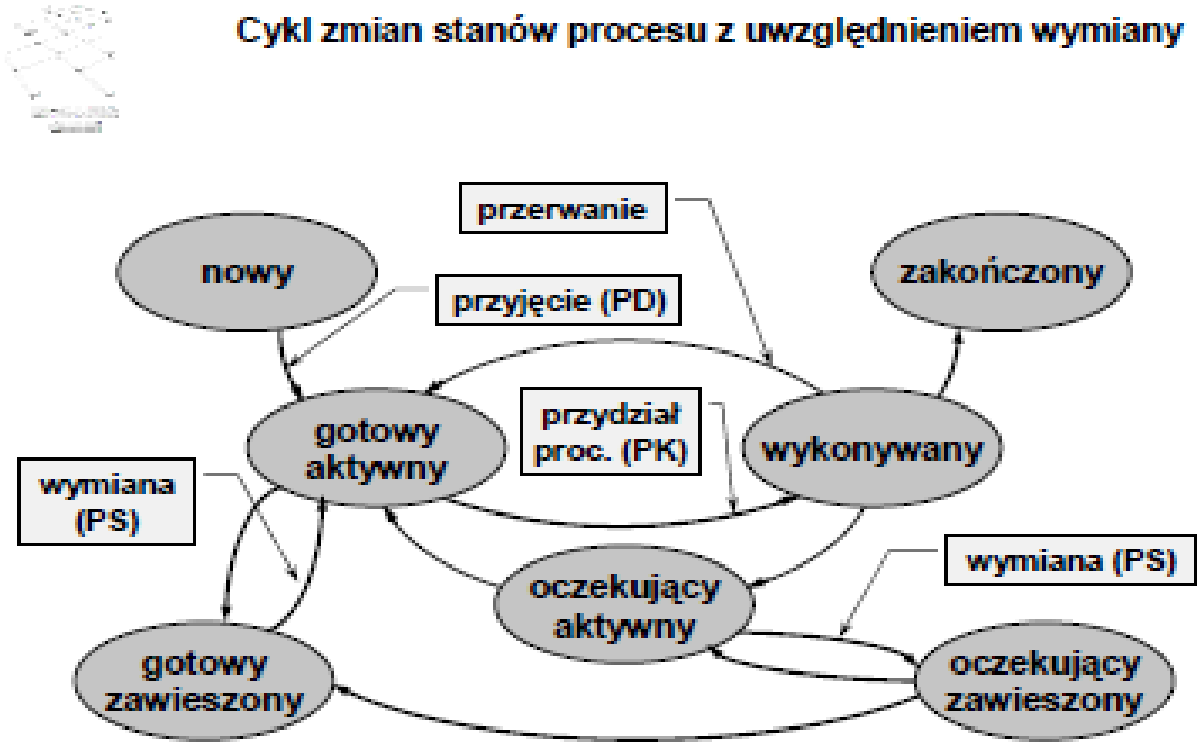
Wymiana procesów w pamięci oznacza usunięcie z pamięci jednego procesu, żeby zasób pamięci oddać innemu procesowi. Umożliwienie wznowienia przetwarzania uwarunkowane jest zapisaniem zwolnionych obszarów pamięci w celu późniejszego odtworzenia. Zapis wykonywany jest na dysku w specjalnie do tego przygotowanym obszarze lub pliku, zwanym *obszarem (plikiem) wymiany*.

Procesy aktywne i zawieszone

Z pamięci usuwany jest najczęściej jakiś proces oczekujący, ale możliwe jest też usunięcie procesu gotowego (zależy to od decyzji planisty). Dla odróżnienia stanów procesu w pamięci operacyjnej od stanów na urządzeniu wymiany (dysk), proces w pamięci określany jest, jako *aktywny*, a proces na urządzeniu wymiany, jako *zawieszony*.

Graf stanu procesów – planiści oraz procesy aktywne i zawieszone

W grafie zmian stanów procesu, uwzględniono stany wynikające z wymiany oraz role planistów.



Decyzję o przyjęciu nowego procesu do systemu podejmuje planista długoterminowy (PD).

Przejście ze stanu gotowy do wykonywany wynika z decyzji planisty krótkoterminowego (PK).

Planista średnioterminowy (PS) odpowiada natomiast za wymianę, czyli decyduje o tym, które procesy usunąć z pamięci, a które ponownie załadować.

Uwzględniając wymianę, można powiedzieć, że pamięć jest zasobem wywłaszczalnym. W przypadku braku wymiany, odebranie procesowi pamięci oznaczałoby jego usunięcie — pamięć byłaby, więc zasobem niewywłaszczalnym.

Problem analizy i oceny efektywności różnych algorytmów planowania dostępu do procesora może być przedmiotem projektów studenckich.

2.3. Zarządzanie wątkami

http://wazniak.mimuw.edu.pl/index.php - systemy operacyjne, sieci komputerowe.

Wątek (lekki proces, ang. lightweight process — LWP), wyróżniony w obrębie procesu ciężkiego (heavyweight), posiadający własne sterowanie i współdzielący z innymi wątkami tego procesu przydzielone (procesowi) zasoby:

– segment kodu i segment danych w pamięci,
– tablicę otwartych plików,
– tablicę sygnałów.

Wątki realizują wyróżnione części programu wykonywanego w obrębie jednego procesu. W jednym procesie może istnieć wiele wątków, które mogą wykonywać niezależne fragmenty programu głównego w dowolnej kolejności lub nawet współbieżnie, w miarę dostępnych zasobów.

Koncepcja wątku (ang. thread) wiąże się ze współdzieleniem zasobów. Każdy proces (ciężki proces w odróżnieniu od lekkiego procesu, czyli wątku) otrzymuje zasoby od odpowiedniego zarządcy i utrzymuje je do swojej dyspozycji.

Zasoby przydzielone procesowi wykorzystywane są na potrzeby sekwencyjnego wykonania programu, ale w wyniku wykonania programu mogą się pojawić kolejne żądania zasobowe. Niedostępność żądanego zasobu powoduje zablokowanie procesu (wejście w stan oczekiwania). W programie wykonywanym przez proces może istnieć jednak inny ***niezależny*** fragment do wykonania, którego żądany zasób nie jest potrzebny. W tym przypadku można by, zatem zmienić kolejność instrukcji w programie i wykonać ten niezależny fragment wcześniej, o ile dostępne są zasoby niezbędne do jego wykonania.

Cechy wątków:

- Wątki wymagają mniej zasobów do działania i też mniejszy jest czas ich tworzenia;

- Wątki korzystają głównie z zasobów przydzielonych procesowi — współdzielą je z innymi wątkami tego procesu.

- Zasobem, o który wątek rywalizuje z innymi wątkami, jest procesor, co wynika z faktu, że jest on odpowiedzialny za wykonanie fragmentu programu.

- Wątek ma, więc własne sterowanie, w związku z tym, *kontekst* każdego wątku obejmuje licznik rozkazów, stan rejestrów procesora oraz stos. Każdy wątek musi mieć swój własny stos, gdzie odkładane są adresy powrotów z podprogramów oraz alokowane są lokalne zmienne.

- Dzięki współdzieleniu przestrzeni adresowej (pamięci) wątki jednego zadania mogą się między sobą komunikować w sposób, który nie wymaga udziału ze strony systemu operacyjnego, np. przekazanie dowolnie dużej ilości danych wymaga przesłania jedynie wskaźnika, zaś odczyt (a niekiedy zapis) danych o rozmiarze nie większym od słowa maszynowego nie wymaga synchronizacji (procesor gwarantuje atomowość takiej operacji).

- Problemem jest jednak właściwa synchronizacja współbieżnie wykonywanych wątków, które korzystają ze wspólnych zasobów. Mogą się pojawić problemy związane z niespójnością wspólnie zapisywanych i odczytywanych danych, prowadzące do błędnego działania programu, podobne do problemów z działaniem transakcji w bazach danych.

Sposoby implementacji wątków

- Realizacja wątków na poziomie jądra systemu operacyjnego — jądro tworzy odpowiednie struktury (blok kontrolny) do utrzymywania stanu wątku.

- Realizacja wątków na poziomie użytkownika — struktury związane ze stanami wątków tworzone są w przestrzeni adresowej procesu.

Wątki współdzielą zasoby, przestrzeń adresową, otwarte pliki oraz sygnały w ramach tego samego procesu, stąd przełączanie kontekstu pomiędzy nimi jest mniej kosztowne, niż w przypadku ciężkich procesów, gdyż wymaga przydziału lub odpowiedniej zmiany stanu znacznie mniejszej liczby zasobów.

Wątki mogą być nawet tak zorganizowane, że jądro nie jest świadome ich istnienia. Deskryptory wątków utrzymywane są w pamięci procesu (a nie jądra) i cała obsługa wykonywana jest w trybie użytkownika.

Alternatywą jest zarządzanie wątkami w trybie systemowym przez jądro, które utrzymuje deskryptory i odpowiada za przełączanie kontekstu pomiędzy wątkami.

Realizacja wątków na poziomie jądra systemu operacyjnego

W przypadku obsługi wątków na poziomie jądra systemu operacyjnego każdy wątek posiada własny blok kontrolny w jądrze systemu, obejmujący:

– stan licznika rozkazów,
– stan rejestrów procesora,
– stan rejestrów związanych z organizacją stosu.

• Własności realizacji wątków na poziomie jądra:

– przełączanie kontekstu pomiędzy wątkami przez jądro,
– większy koszt przełączania kontekstu,
– bardziej sprawiedliwy przydział czasu procesora.

Obsługa wielowątkowości na poziomie jądra systemu (w trybie systemowym) oznacza, że wszelkie odwołania do mechanizmów obsługi wątków wymagają dostępu do usług jądra, co zwiększa koszt czasowy realizacji.

Jądro musi też utrzymać bloki kontrolne (deskryptory) wątków, co w przypadku wykorzystania statycznych tablic może stanowić istotny koszt pamięciowy.

Z drugiej strony, świadomość istnienia wątków procesu umożliwia uwzględnienie tego faktu w zarządzaniu zasobami przez jądro i prowadzi do poprawy ich wykorzystania.

Realizacja wątków w trybie użytkownika

W przypadku wątków obsługiwanych na poziomie procesów użytkownika deskryptor wątku znajduje się w tablicy wątków w pamięci danego procesu (jądro nie wie nic o wątkach).

Własności realizacji wątków na poziomie użytkownika:
- przydział czasu procesora dla procesu (nie dla wątku);
- przełączanie kontekstu pomiędzy wątkami przez jawne odwołania do mechanizmu obsługi wątków;
- mniejszy koszt przełączania kontekstu (bez angażowania jądra systemu operacyjnego);
- możliwość „głodzenia” wątków tego samego procesu, gdy jeden z nich spowoduje przejście procesu w stan oczekiwania.

Realizacja wątków przez odpowiednią bibliotekę w trybie użytkownika zwiększa szybkość przełączania kontekstu, ale powoduje, że jądro, nie wiedząc nic o wątkach, planuje przydział czasu procesora dla procesów. Oznacza to, że w przypadku większej liczby wątków procesu czas procesora, przypadający na jeden wątek jest mniejszy, niż w przypadku procesu z mniejszą liczbą wątków.

Problemem jest też wprowadzanie procesu w stan oczekiwania, gdy jeden z wątków zażąda operacji wejścia-wyjścia lub utknie na jakimś mechanizmie synchronizacji z innymi procesami. Planista traktuje taki proces, jako oczekujący do czasu zakończenia operacji, podczas gdy inne wątki, o których jądro nie wie, mogłyby się wykonywać.

Obsługa wątków w wybranych systemach operacyjnych

W niektórych systemach operacyjnych wyróżnia się zarówno wątki trybu użytkownika, jak i wątki trybu jądra.

W systemie Solaris Unix terminem *wątek* określa się wątek, istniejący w trybie użytkownika, a wątek trybu jądra określa się, jako *lekki proces*.

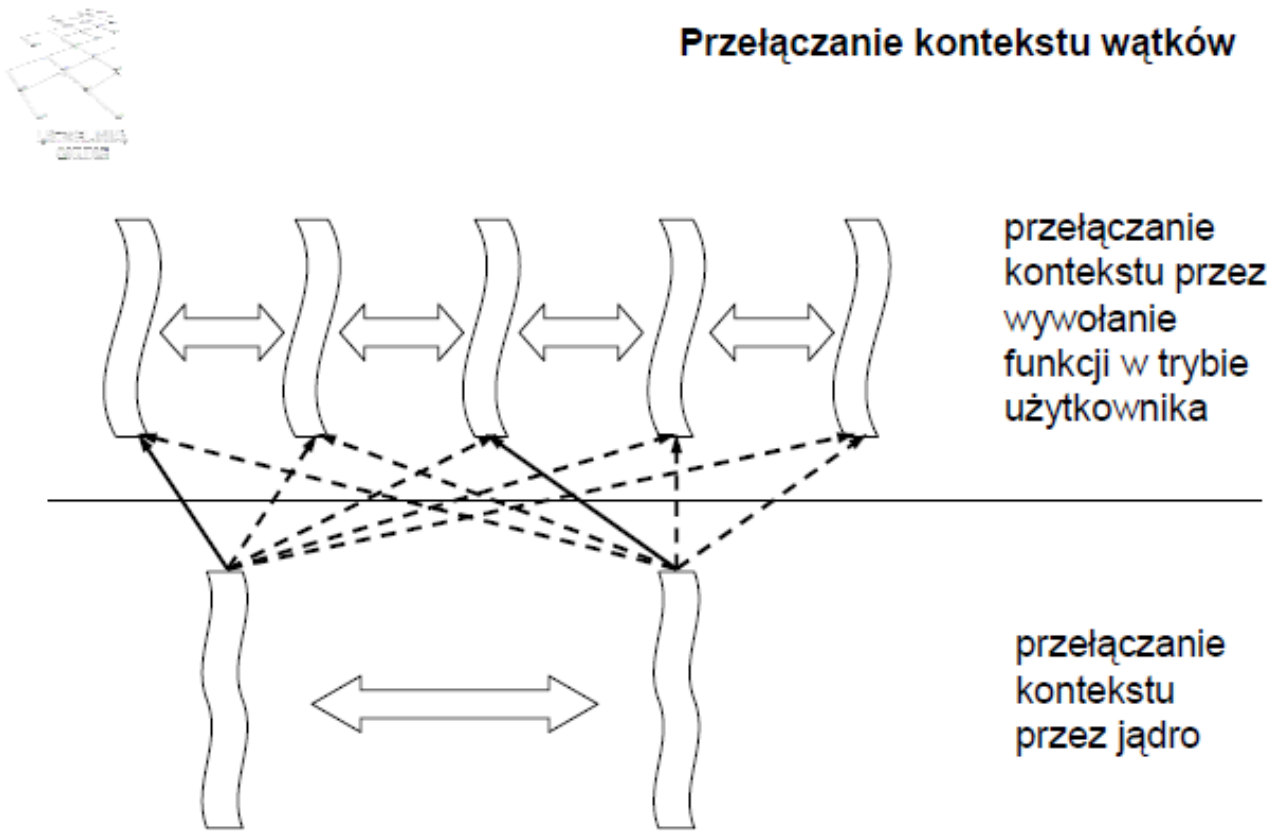
W systemie Windows wprowadza się pojecie *włókna*, zwanego też *lekkim wątkiem* (ang. fiber, lightweight thread), które odpowiada wątkowi trybu użytkownika, podczas gdy termin wątek (lekki proces) odnosi się do wątku trybu jądra. Takie rozróżnienie umożliwia operowanie pewną liczbą wątków trybu jądra, a w ramach realizowanych przez te wątki programów może następować przełączanie pomiędzy różnymi wątkami trybu użytkownika bez wiedzy jądra systemu. Wątek trybu jądra można, więc traktować, jako wirtualny procesor dla wątku trybu użytkownika.

Przełączanie kontekstu lekkich procesów (wątków) jądra

Kontekst pomiędzy dwoma lekkimi procesami (wątkami) przełączany jest przez jądro.

Każdy z lekkich procesów wykonuje jakiś wątek trybu użytkownika (włókno), co obrazuje ciągła linia ze strzałką.

Dla każdego lekkiego procesu istnieje, zatem bieżące włókno. W ramach wykonywanego kodu takiego włókna może nastąpić wywołanie funkcji zachowania bieżącego kontekstu, a następnie funkcji odtworzenia innego (wcześniej zachowanego) kontekstu (włókna), o ile tylko w miejscu wywołania dostępny jest odpowiedni deskryptor, opisujący odtwarzany kontekst. Potencjalnie, więc każdy z lekkich procesów może wykonywać dowolne z włókien, co symbolizuje przerywana linia.



W systemach operacyjnych istnieją odpowiednie procedury umożliwiające zarządzanie wątkami:

- tworzenie wątku,
- usuwanie wątku,
- wstrzymywanie i wznawianie wątku,
- zmianę priorytetu wątku,
- oczekiwanie na zakończenie wątku.

Realizacja procesów/wątków w systemie Linux

W jądrze systemu Linux nie odróżnia się pojęcia wątku od procesu.

- Procesy mogą współdzielić takie zasoby, jak:

– przestrzeń adresowa,
– otwarte pliki,
– informacje o systemie plików,
– procedury obsługi sygnałów.

- Deskryptory procesów (o strukturze struct task_struct) przechowywane są na dwukierunkowej, cyklicznej liście zadań.

Proces potomny tworzony jest w systemie Linux poprzez wywołanie funkcji *clone*.

Funkcja ta wykorzystywana jest między innymi do implementacji funkcji *fork*, ujętej w standardzie POSIX (Portable Operating System Interface for Unix).

Tworząc nowy proces z użyciem funkcji clone można określić, które zasoby procesu macierzystego mają być współdzielone z potomkiem.

W zależności od zakresu współdzielonych zasobów, nowo utworzony proces *może być* uznawany za *wątek* lub za *ciężki proces*. Typowe wątki będą współdzielić przestrzeń adresową, otwarte pliki i inne informacje związane z systemem plików (np. katalog bieżący, korzeń drzewa katalogów) i procedury obsługi sygnałów.

Rozróżnienie proces ciężki – proces lekki sprowadza się, zatem do określenia zakresu współdzielenia zasobów. Proces ciężki współdzieli zasoby procesu macierzystego.

Stany procesu (wątku) w systemie Linux

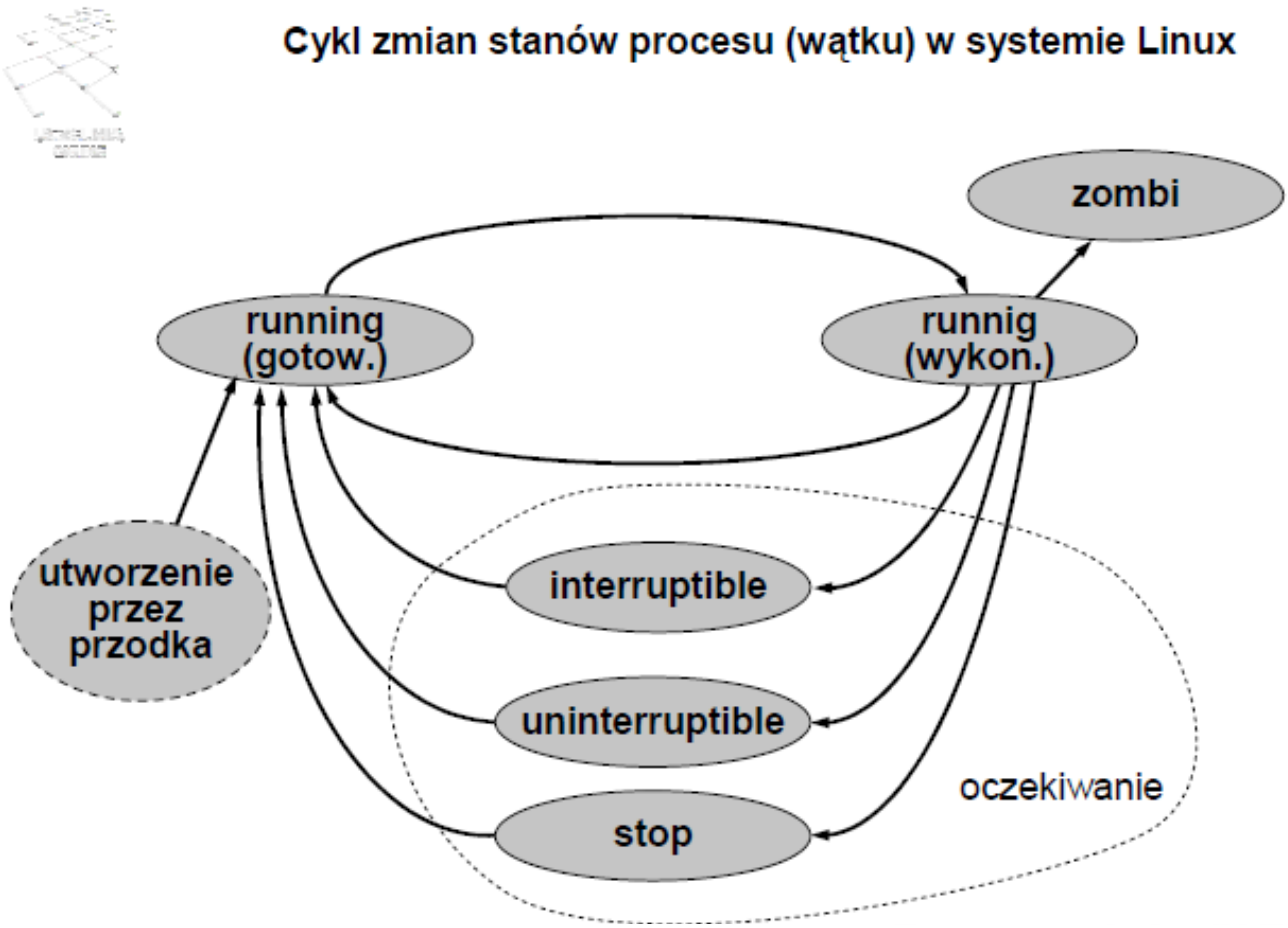
TASK_RUNNING — wykonywanie lub gotowość (do wykonania);

TASK_INTERRUPTIBLE — oczekiwanie na zajście zdarzenia lub sygnał;

TASK_UNINTERRUPTIBLE — oczekiwanie na zajście zdarzenia, przy czym sygnały są ignorowane;

TASK_ZOMBI — stan zakończenia utrzymywany w celu przechowania deskryptora procesu;

TASK_STOP — zatrzymanie w wyniku otrzymania sygnału (np. SIGSTOP).



Cykl zmian stanów w Linuksie jest bardzo prosty — odpowiada dość dokładnie ogólnemu schematowi. Jedyna różnica to wyodrębnienie dwóch stanów oczekiwania — w jednym następuje reakcja na sygnały (interruptible), w drugim sygnały są ignorowane (uninterruptible).

Jako specyficzny rodzaj oczekiwania można też traktować stan wstrzymania TASK_STOP.

Specyficzną cechą jest również brak rozróżnienia pomiędzy stanem gotowości a stanem wykonywania. Są to oczywiście dwa różne stany, ale w deskryptorze procesu oznaczone w taki sam sposób.

Procesy w systemie Windows 2000/XP

- Proces stanowi środowisko do wykonywania wątków.

- Struktury opisu procesu obejmują:

– EPROCESS — blok centrum wykonawczego, opisujący proces,

– KPROCESS — blok kontrolny procesu, część struktury EPROCESS,

– PEB — blok środowiska procesu, dostępny w trybie użytkownika.

Proces w systemie Windows gromadzi zasoby na potrzeby wykonywania wątków, wchodzących w jego skład.

Informacje o procesie znajdują się w strukturze EPROCESS, której częścią jest właściwy blok kontrolny (KPROCESS).

Zawartość obu tych struktur dostępna jest w trybie jądra. W ich skład wchodzi wiele wskaźników do innych struktur (między innymi struktur opisujących wątki).

Część opisu procesu — blok środowiska procesu PEB — znajduje się w części przestrzeni adresowej, dostępnej w trybie użytkownika.

Wątki w systemie Windows XP

- Wątki korzystają z zasobów przydzielonych procesom.

- Wątki (nie procesy) ubiegają się o przydział procesora i są szeregowane przez planistę krótkoterminowego.

- Struktury opisu wątku obejmują:
  – ETHREAD — blok centrum wykonawczego, opisujący wątek,
  – KTHREAD — blok kontrolny procesu, część struktury ETHREAD,
  – TEB — blok środowiska procesu, dostępny w trybie użytkownika.

Podstawowe zasoby na potrzeby wykonania wątku (np. pamięć) przydzielone są procesowi. Są one, zatem wspólne dla wszystkich wątków danego procesu.

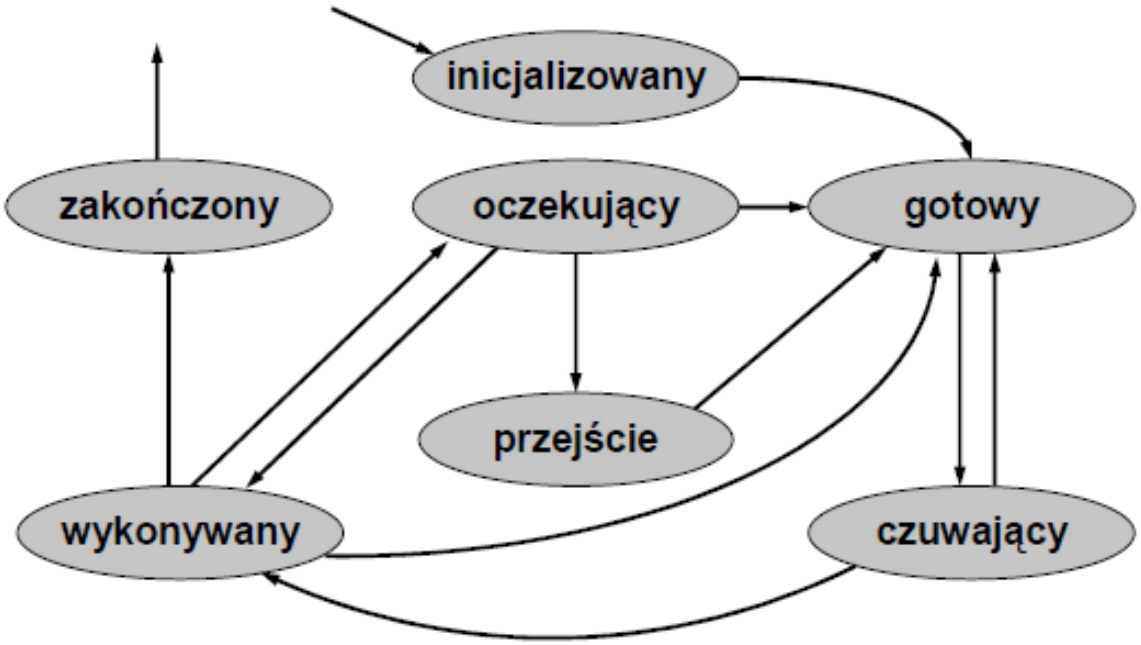
Najważniejszym zasobem przydzielanym wątkowi jest procesor.

Wszelkie przetwarzanie i wynikająca stąd zmiana stanu procesu odbywa się w wątku. Struktury opisu wątku są analogiczne do struktur opisu procesu.

Stany wątku w systemie Windows XP

- Inicjalizowany (initialized, wartość 0) — stan wewnętrzny w trakcie tworzenia wątku,
- Gotowy (ready, wartość 1) — oczekuje na przydział procesora,
- Wykonywany (running, wartość 2),
- Czuwający (standby, wartość 3) — wybrany do wykonania, jako następny,
- Zakończony (terminated, wartość 4),
- Oczekujący (waiting, wart. 5) — oczekuje na zdarzenie,
- Przejście (transition, wartość 6) — oczekuje na sprowadzenie swojego stosu jądra z pliku wymiany,
- Unknown (wart. 7).

Cykl zmian stanów wątku w systemie Windows XP



W systemie Windows zarówno stan gotowości, jak i czuwania odpowiada stanowi gotowości w odniesieniu do ogólnego schematu zmian stanów.

2.4. Modele obliczeń

Procesy i wątki realizowane pod kontrolą systemu operacyjnego mogą się komunikować z wykorzystaniem różnych mechanizmów w celu synchronizacji działania.

Mechanizmy te są związane ze sposobem przetwarzania zadań:

- sekwencyjnym,

- współbieżnym,

- równoległym.

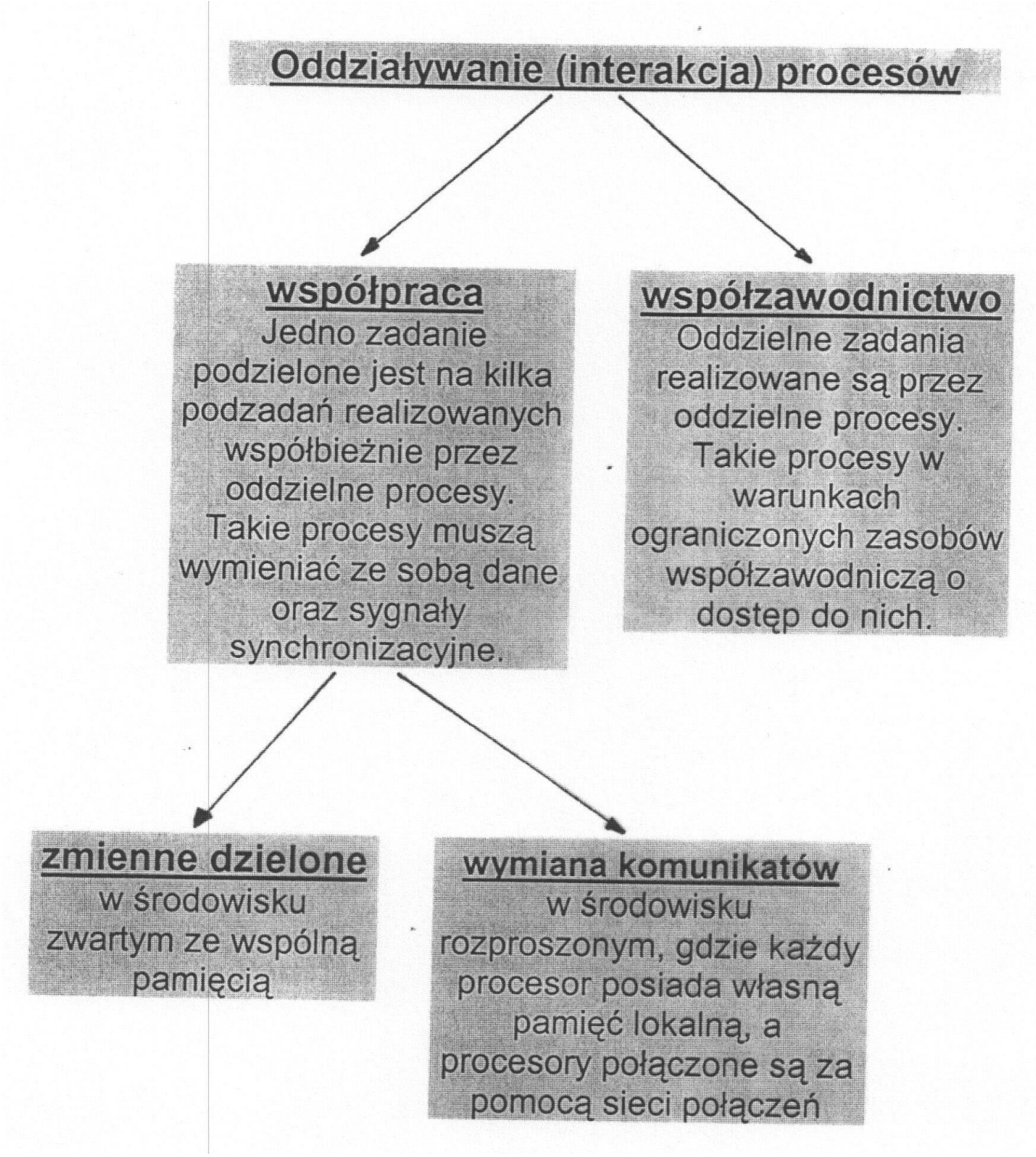
[lit] Zbigniew Czech, Wprowadzenie do obliczeń równoległych, PWN, 2010.

[lit] Kai Hwang, Faye A. Briggs, Computer Architecture and Parallel Processing, McGraw-Hill Int. Ed., 1985.

Interakcja procesów (wątków) może się odbywać na zasadzie:

- współpracy;

- współzawodnictwa.



Metody przetwarzania zadań

- sekwencyjne;

- współbieżne: quasi-równoległe, równoległe.



W przypadku **przetwarzania sekwencyjnego** zadania obliczeniowe są wykonywane kolejno po sobie na jednym procesorze.

W przypadku **przetwarzania współbieżnego (quasi-równoległego)** zadania obliczeniowe są wykonywane na jednej jednostce przetwarzającej, ale z podziałem czasu.

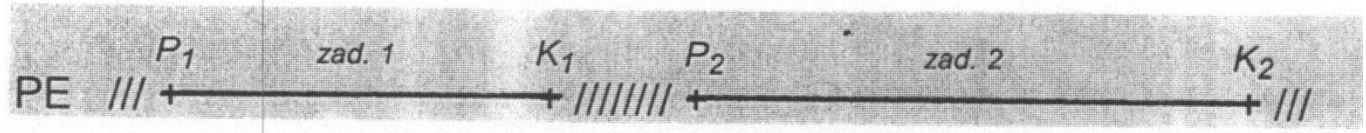
Można postawić pytanie:

Kiedy i pod jakimi warunkami przetwarzanie n zadań sekwencyjnie jest wolniejsze od przetwarzania współbieżnego n zadań np. z użyciem n – wątków (zależy od czasu trwania – złożoności pojedynczego zadania oraz liczby zadań).

Przetwarzanie sekwencyjne

2 zadania obliczeniowe

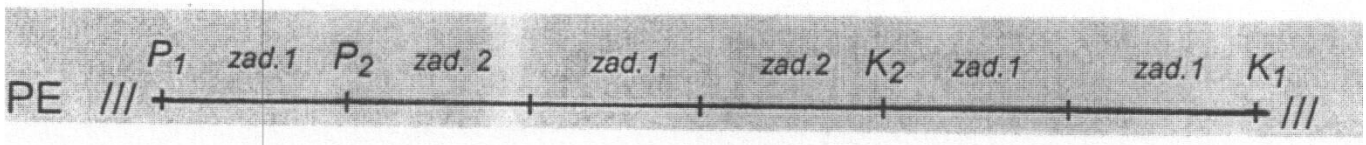
1 jednostka przetwarzająca (PE)



Przetwarzanie współbieżne – quasi-równoległe

2 zadania obliczeniowe

1 jednostka przetwarzająca z podziałem czasu (PE)

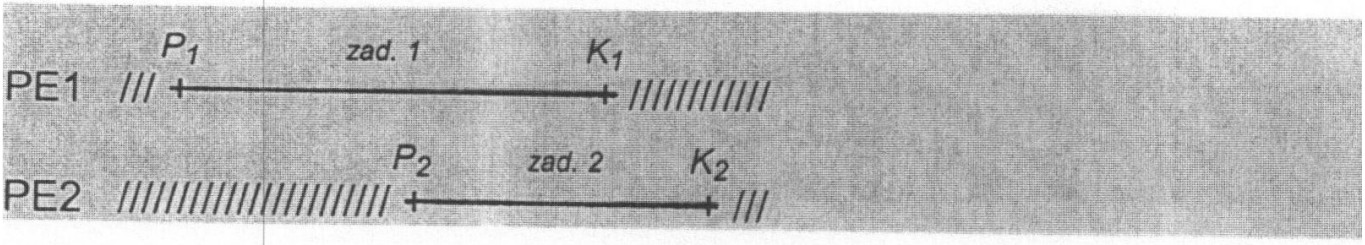


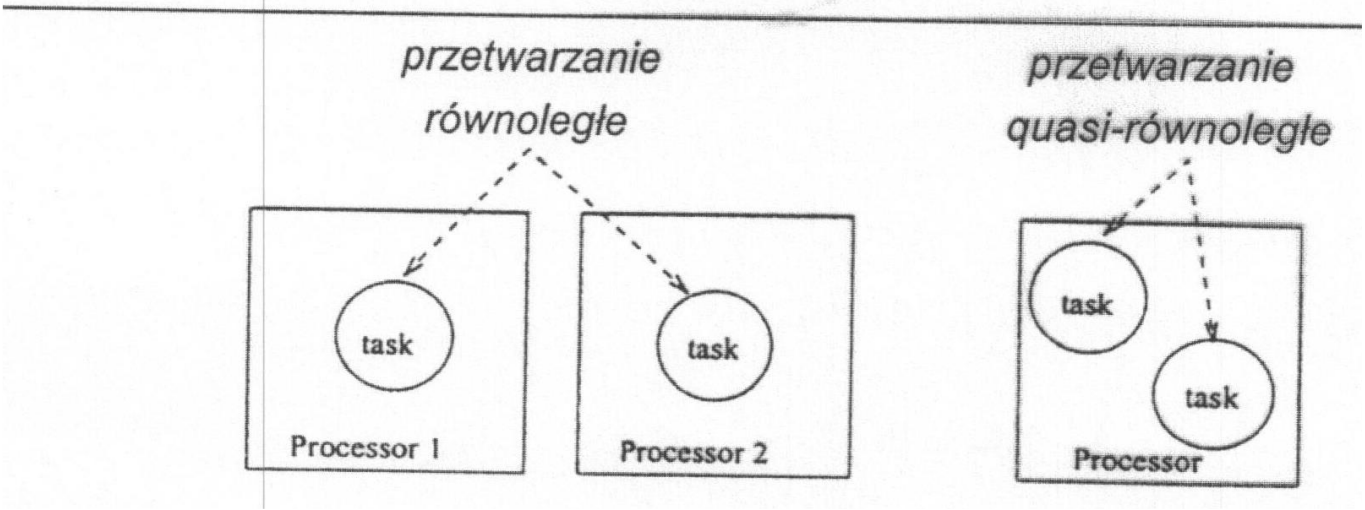
W przypadku **przetwarzania współbieżnego - równoległego** każde zadanie obliczeniowe jest wykonywane na odrębnej jednostce przetwarzającej.

Przetwarzanie współbieżne – równoległe

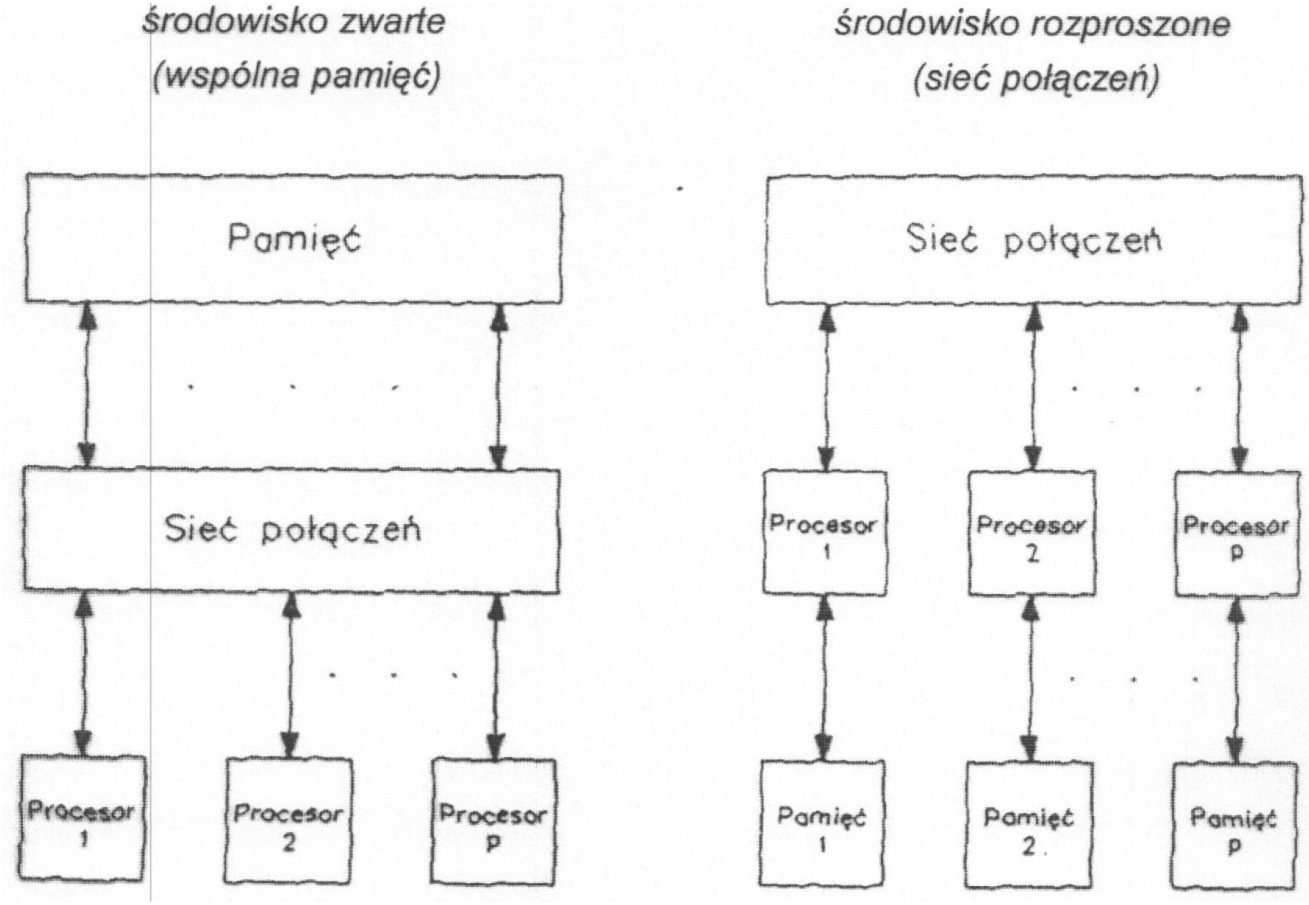
2 zadania obliczeniowe

2 jednostki przetwarzające (PE1, PE2)





Przetwarzanie współbieżne może odbywać się w środowisku zwartym lub rozproszonym.

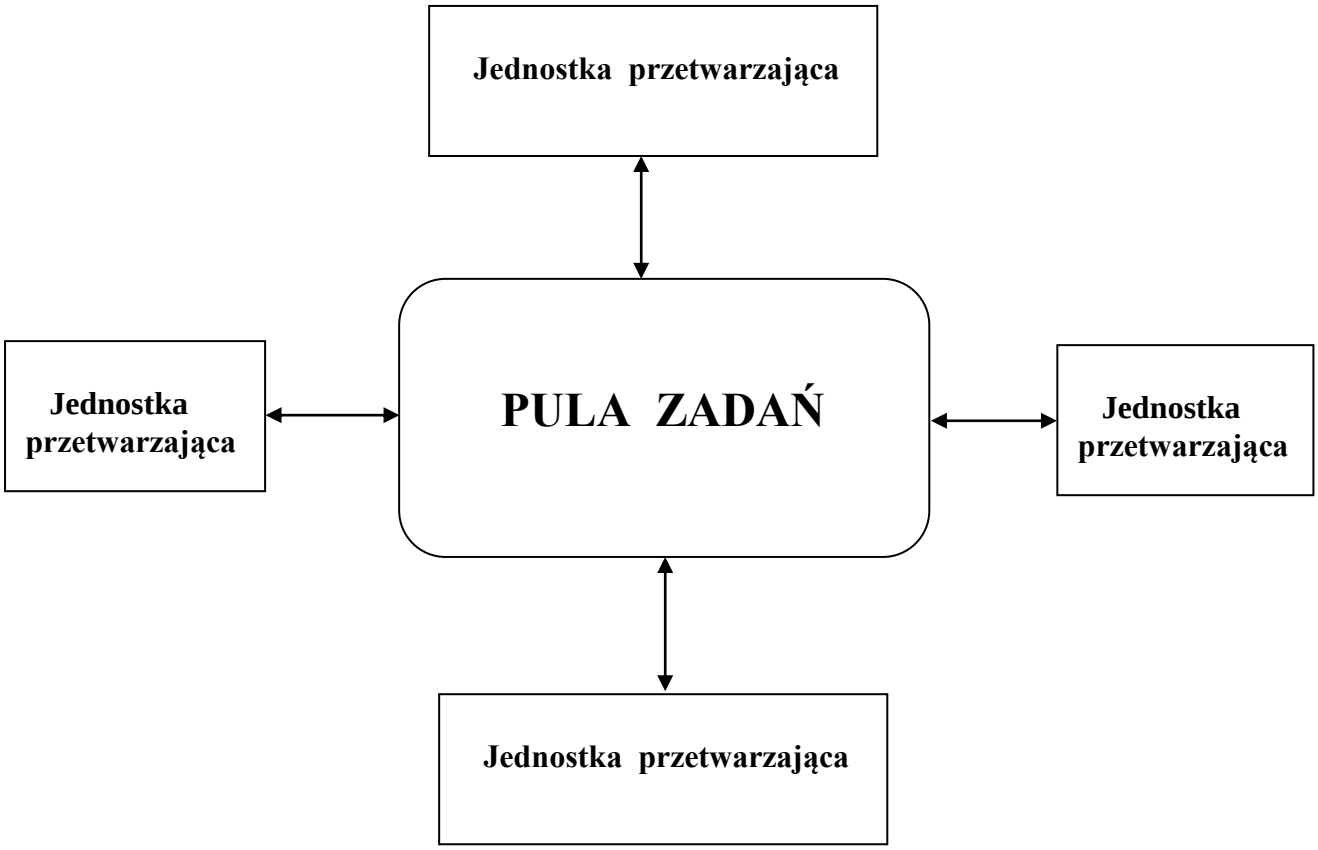


Zadania obliczeniowe mogą być przetwarzane w sposób:

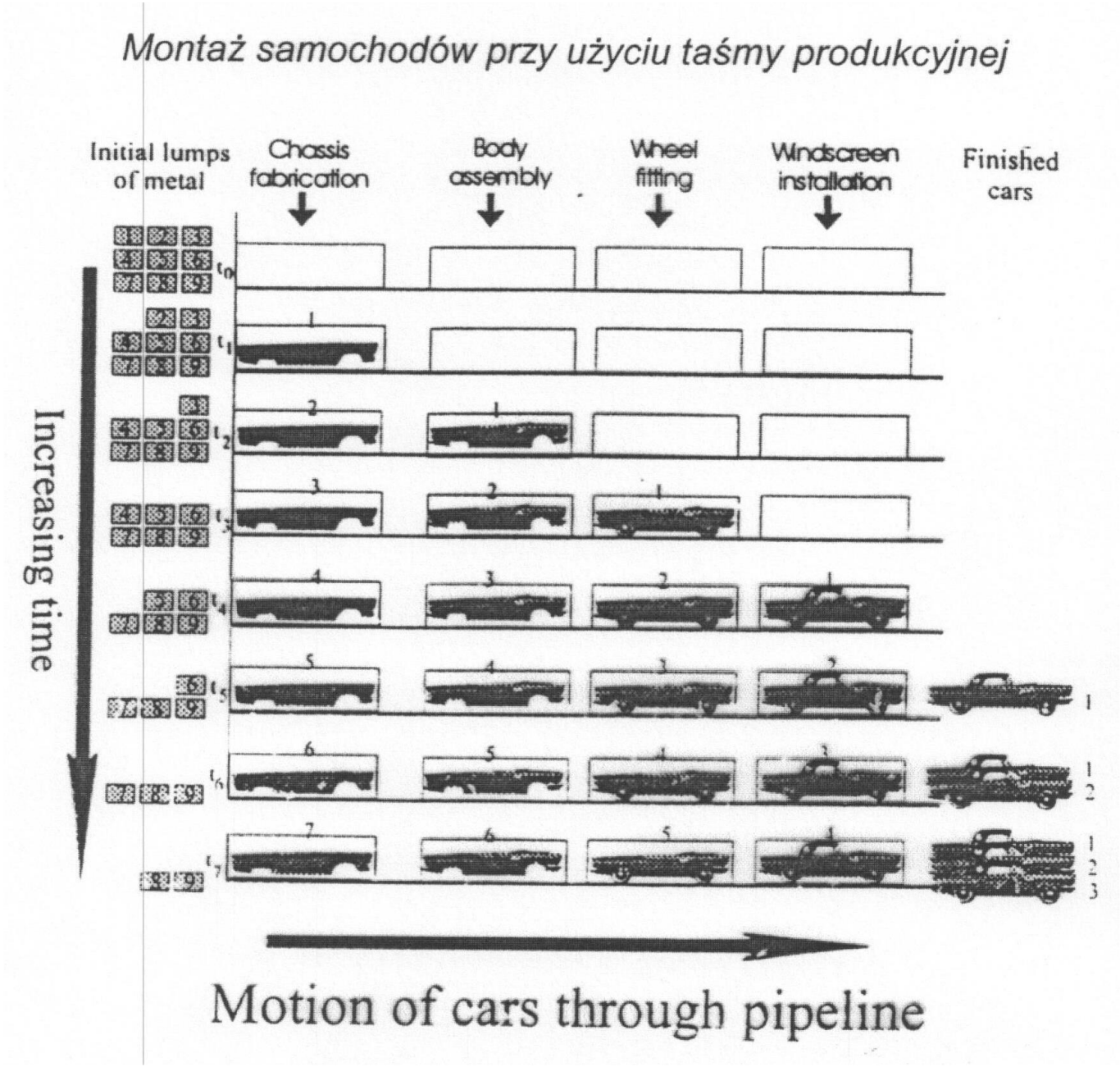
- współbieżny przestrzennie przez kilka niezależnych jednostek;

- współbieżny czasowo (potokowy) przez kilka powiązanych jednostek; zadania znajdują się w różnych fazach wykonania.

Współbieżność przestrzenna



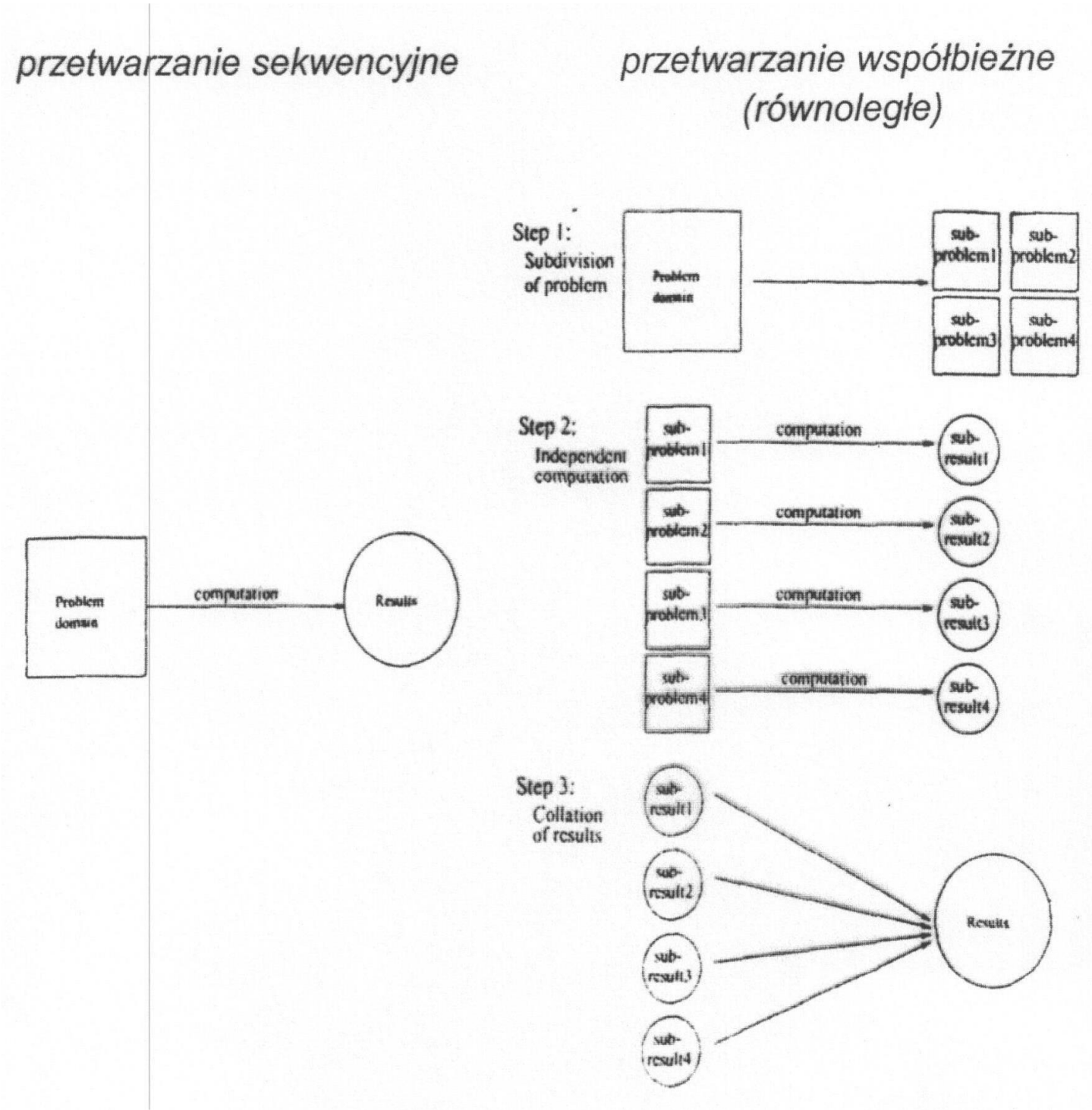
Współbieżność czasowa – różne fazy wykonania zadań



W danej chwili czasowej t_1, t_2, itd., kilka zadań jest realizowanych współbieżnie (równolegle) przy czym część z nich jest na innym etapie, a niektóre są zakończone.

W przypadku, gdy dana jednostka przetwarzająca jest w 100% obciążona pojawia się tzw. „wąskie gardło”.

Sterowanie w przetwarzaniu sekwencyjnym i współbieżnym



Poziomy przetwarzania i programowania równoległego

Równoległość może być wprowadzana na różnych poziomach:

- poziom prac (pracami są całe programy lub ich ciągi);

- poziom zadań (zadaniami są procedury lub iteracje pętli);

- poziom instrukcji (rozkazów);

- poziom operacji wewnątrz rozkazu.

1. Poziom prac

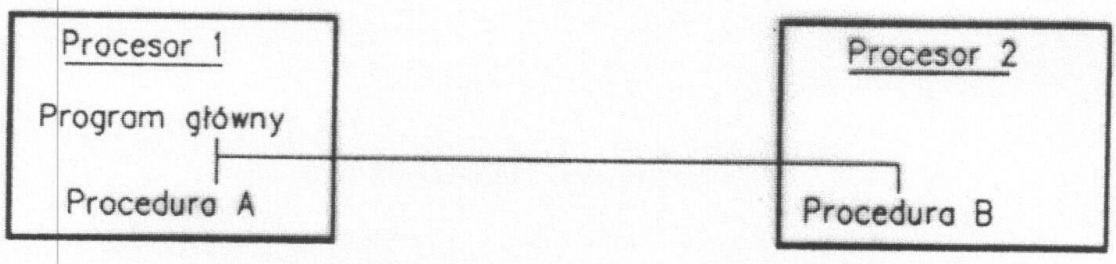
Równoczesna realizacja niezależnych prac (ang. job). Pracami są programy lub ich ciągi.



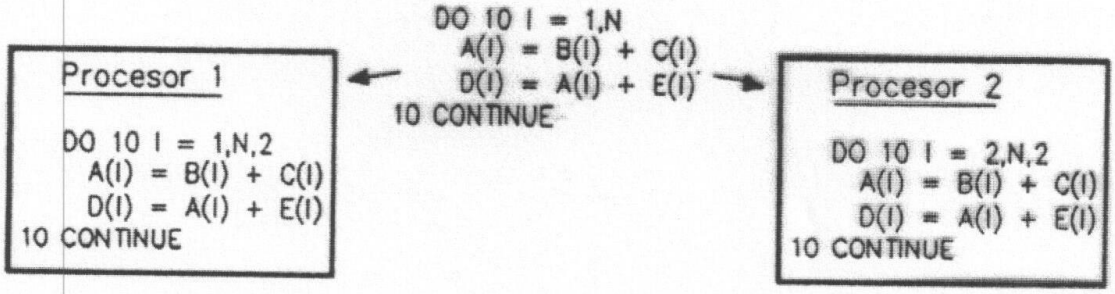
2. Poziom zadań

Program podlega dekompozycji na niezależne zadania (ang. task) wykonywane równolegle. Zadaniami są wydzielone fragmenty programu, najczęściej w formie procedur (ang. macrotasking) lub niezależnych iteracji pętli programowych (ang. microtasking).

a)



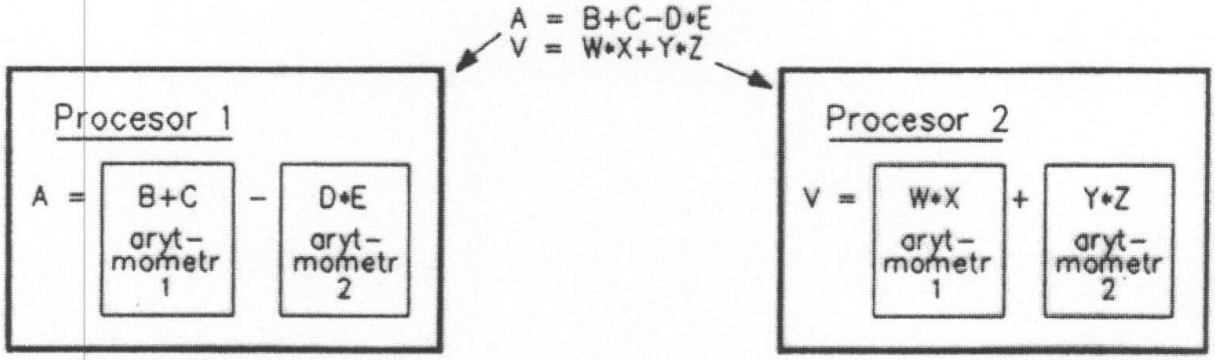
b)



(a) zadaniami są procedury; (b) zadaniami są iteracje pętli

3. Poziom instrukcji (rozkazów)

Równolegle wykonywane są niezależne instrukcje programu (niezależne rozkazy w ramach instrukcji).



4. Poziom operacji wewnątrz rozkazu

Fazy rozkazu, które są niezależne, wykonywane są równolegle (potokowe wykonywanie rozkazów)

W przedstawionej klasyfikacji, im niższy poziom przetwarzania tym większa rola mechanizmów sprzętowych w realizacji przetwarzania równoległego. Jednocześnie, im wyższy poziom, tym istotniejsze stają się mechanizmy programowe, zanika natomiast zależność od sprzętu. Przykładowo, poziom 4. jest praktycznie niedostępny dla programisty, cała równoległość jest zapewniona sprzętowo. Z kolei, poziom 1. można realizować zarówno w systemach wieloprocesorowych , gdzie każdemu procesorowi przydziela się oddzielny program, jaki i w jednoprocesorowych, korzystając z pracy wieloprogramowej i podziału czasu.

Zastosowanie przetwarzania równoległego prowadzi do przyspieszenia obliczeń, ale poziom tego przyspieszenia zależy od udziału części sekwencyjnej w algorytmie zrównoleglonym.

Poziom przyspieszenia obliczeń uzyskany w algorytmie równoległym, wykorzystującym p procesorów wyraża prawo Amdahla.

Prawo Amdahla

Założenia

T_1 – czas wykonania najszybszego algorytmu sekwencyjnego na pojedynczym procesorze

T_p – czas wykonania algorytmu równoległego na p procesorach

$A_p = \frac{T_1}{T_p}$ – przyspieszenie

s - część algorytmu sekwencyjnego, która musi być wykonana sekwencyjnie (nie można jej zrównoleglić)

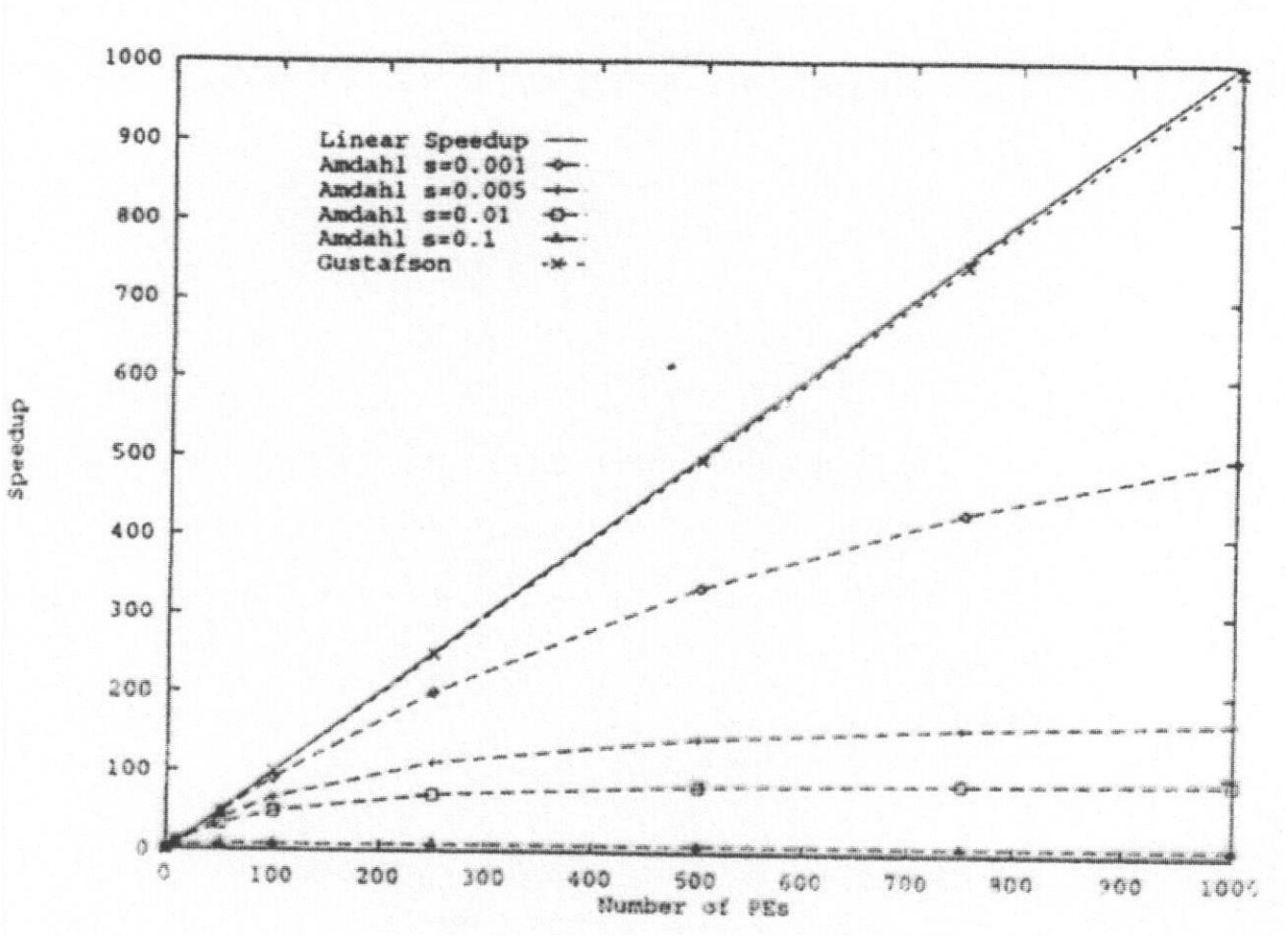
$$T_p = sT_1 + \frac{(1-s)T_1}{p}$$

Prawo Amdahla

$$A_p = \frac{p}{1+(p-1)s}$$

Wykres zależności przyspieszenia Ap od udziału części sekwencyjnej s.

Im *s* mniejsze tym większe przyspieszenie. Przy $s = 0$ jest $A_p = p$.



Wniosek 1.

$$\lim_{p\to\infty} A_p = \lim_{p\to\infty} \frac{1}{\frac{1}{p} + (1 - \frac{1}{p})s} = \frac{1}{s}$$

Niezależnie od liczby procesorów użytych do wykonania algorytmu równoległego przyspieszenie nie może być większe niż 1/s.

Przykład: dla s = 0.25 i p = 16, przyspieszenie Ap nie przekroczy wartości 4.

Wniosek 2.

Dla większości algorytmów współczynnik **s** zależy od rozmiaru zadania **n** (liczby danych wejściowych).

Algorytm równoległy jest efektywny, jeżeli:

$$\lim_{n\to\infty} s(n) = 0.$$

Zatem

$$\lim_{n\to\infty} A_p = \frac{p}{1+(p-1)s(n)} = p$$

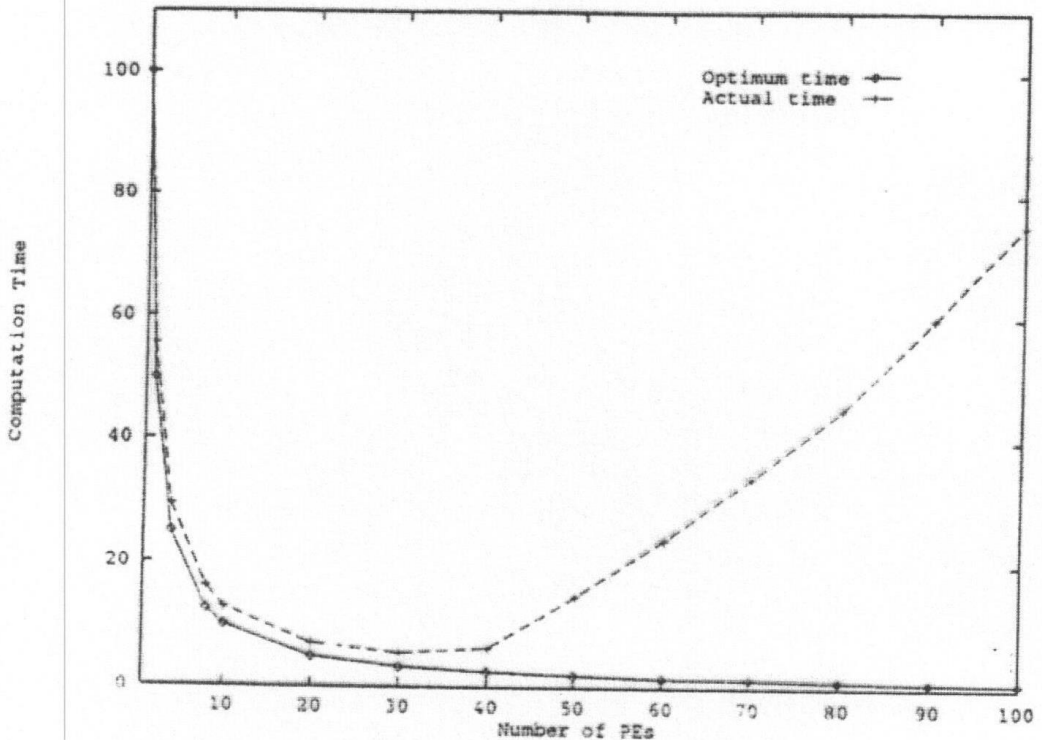
Największe przyspieszenie osiągane przy wykonaniu zadania przez p procesorów nie przekracza p.

Jest ono tym większe im większy jest rozmiar zadania n.

Liczba procesorów a czas wykonania

Zwiększanie liczby procesorów realizujących zadanie powoduje:

- zwiększenie czasu potrzebnego na synchronizację i wymianę danych pomiędzy procesorami,
- zwiększenie czasu zużywanego przez system operacyjny na obsługę procesów.

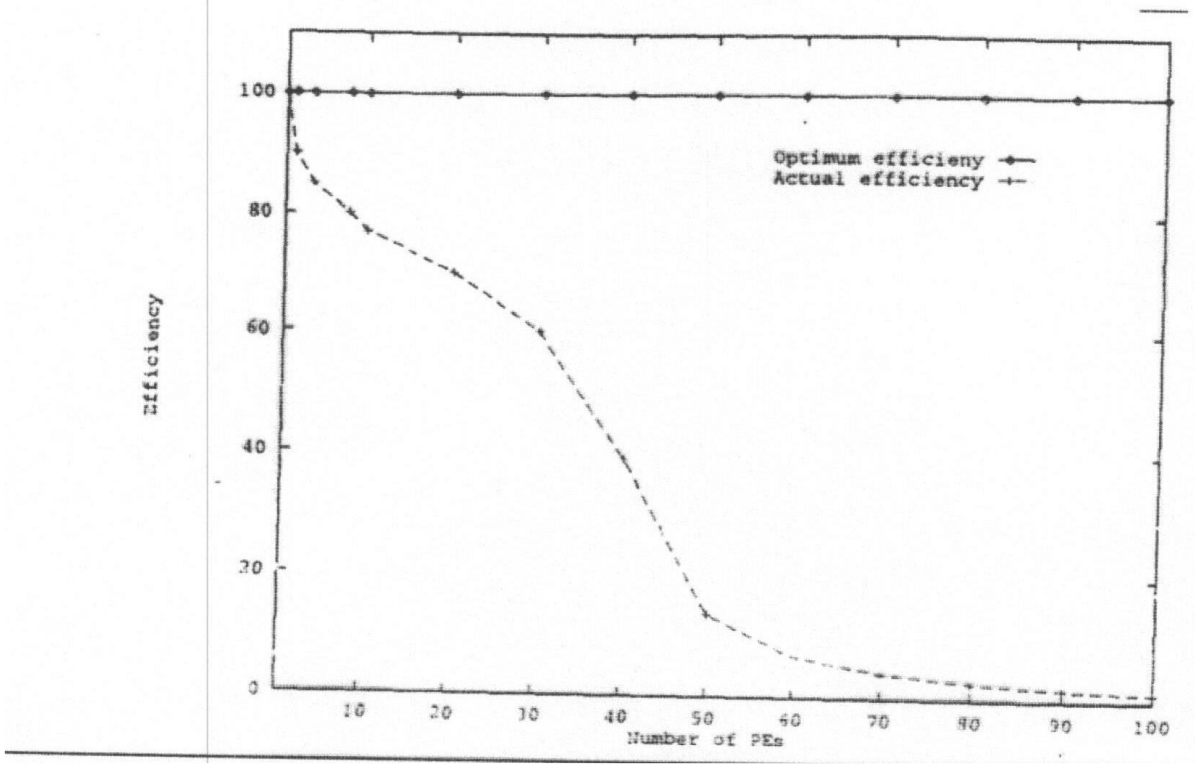


Efektywność wykorzystania procesora

p - liczba procesorów realizujących zadnie

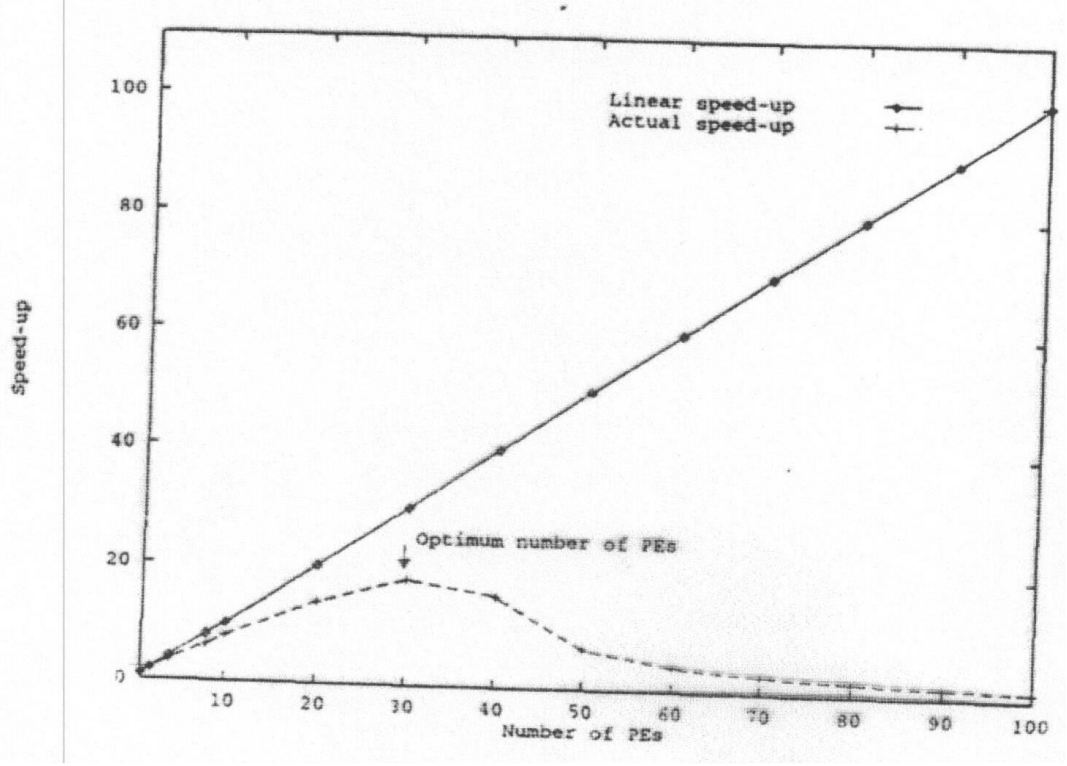
A_p - przyspieszenie otrzymane przy użyciu p procesorów

$e = \frac{A_p}{p}$ - efektywność wykorzystania pojedynczego procesora



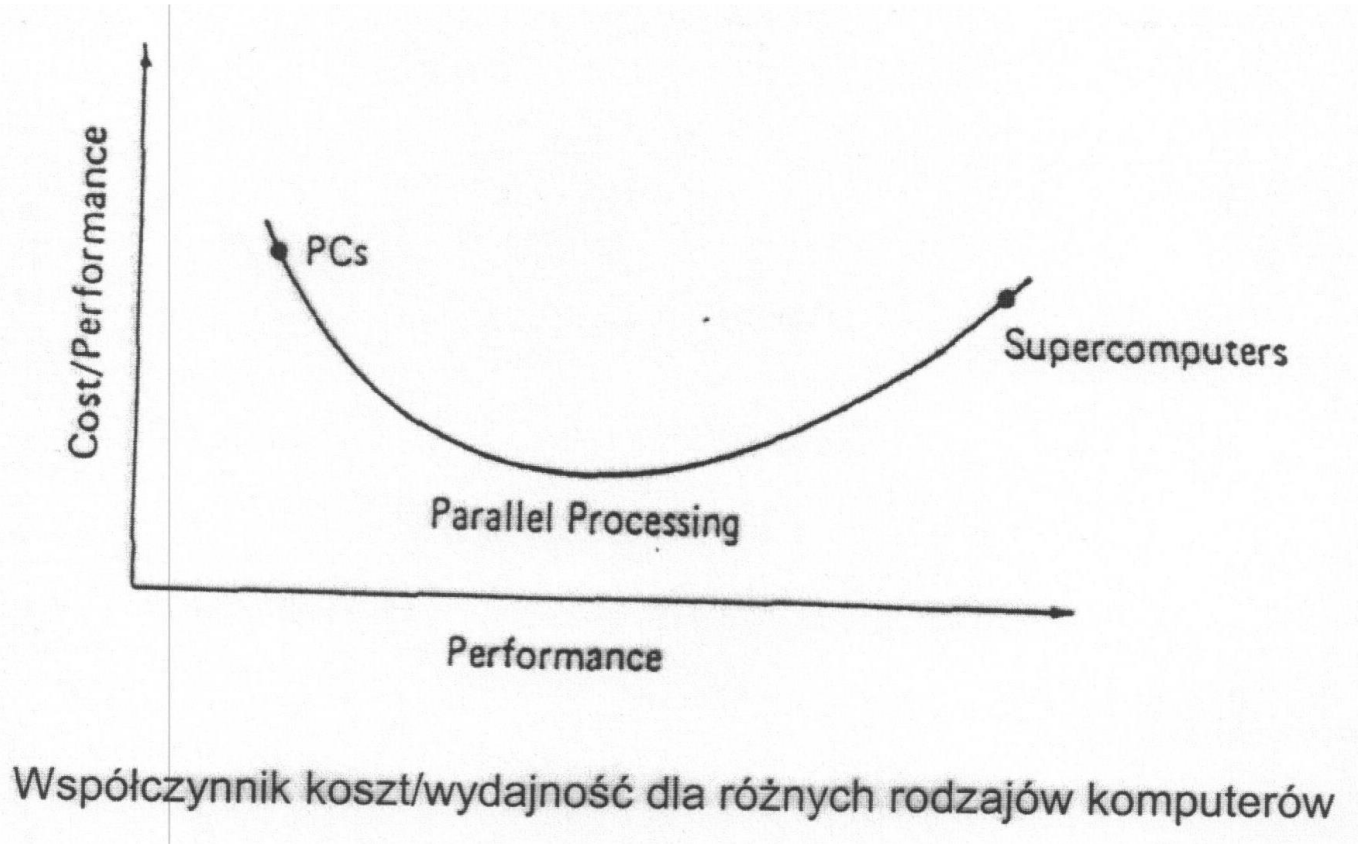
Optymalna liczba procesorów

Dla każdego zadania istnieje optymalna liczba procesorów zaangażowanych w jego rozwiązanie i dająca maksymalne przyspieszenie.



Na przyspieszenie obliczeń ma również wpływ efektywność oprogramowania systemowego (w tym systemu operacyjnego).

Współczynnik koszt/wydajność



Współczynnik koszt/wydajność dla różnych rodzajów komputerów